

2차 보고서

26-1 캡스톤디자인과창업프로젝트B (그로쓰)



팀 번호	37
팀 이름	Optimus Prime
트랙	연구
팀장 및 팀원	강시연 (팀장, 2376008) 김지윤 (팀원, 2176101) 신우림 (팀원, 2376141)
지도교수	반효경

1. Team Info

1.1. 과제명

VM별 AI 워크로드 에너지 사용량 예측모델 설계

1.2. 팀 정보

- 팀 번호:37
- 팀 이름: Optimus Prime
- 트랙: 연구
- 지도교수: 반효경 교수

1.3. 팀 구성원

- 강시연 (2376008): 팀장, 과제제출물 및 면담 일정 관리/Task 분배, *트랙 A: 컴퓨터 자원 이용률과 에너지 소비량 사이의 비선형성 분석* 담당
- 김지윤 (2176101): 팀원, 팀 워크스페이스 (노션) 관리 및 정리, *트랙 B: 다중 워크로드 환경에서 에너지 사용 분리* 담당
- 신우림 (2376141): 팀원, 각종 자료 디자인 및 제작, *트랙 C: 실제 VM 환경으로의 모델 확장 및 검증* 담당

2.Project-Summary

2.1 문제 정의

현재 클라우드 서비스의 과금 시스템은 실제 에너지 소비가 아닌, CPU 코어 수·메모리 용량·GPU 유형 등 자원 할당에 기반하고 있다. 그러나 대규모 AI 워크로드 환경에서는 이 구조가 다음 세 가지 문제를 야기한다.

- 동일한 자원을 할당받아도 워크로드에 따라 실제 전력 소비가 수 배 차이난다. 예를 들어 경량 서버 프로세스(NodeJS, ~79 W)와 GPU 행렬 연산(GEMM, ~278 W)은 유사한 논리적 자원을 점유하지만 에너지 소비량은 크게 다르다.
- Multi-Tenant 환경에서 동시 실행 워크로드 간 자원 비율을 재분배해도 전체 시스템 전력은 거의 변하지 않는다. 즉, 자원 할당 비율은 에너지 비용의 신뢰할 수 있는 대리 지표(proxy)가 아니다.
- GPU 지배적인 AI 워크로드는 CPU 중심 전통 모델과 에너지 소비 구조가 근본적으로 다르며, 자원 유형(CPU·GPU·메모리·스토리지)마다 에너지 행동 특성도 상이하다.

따라서 본 연구는 보다 공정한 과금 체계를 위해 에너지 사용량 기준 과금을 제안하고, 워크로드 특성을 반영한 정확한 에너지 측정 모델을 제시한다.

2.2 기존 연구와의 비교

아래 표는 기존 연구와 본 연구를 5가지 측면에서 비교한 것이다. '주요 비교 대상 연구'에 표를 작성하는데 근거한 기존 연구들의 목록 형태로 작성하였다.

구분	기존 연구	본 연구
접근 방식	에너지모델링(수식기반), DVFS/스케줄링 최적화; CPU 전력 예측에 집중하여 하드웨어 카운터 또는 선형 사용률 모델 활용	(Track A,B,C) 실측 기반 실증 분석; 플랫폼 내장 통계만 활용(RAPL, nvidia-smi, cgroup)
대상 환경	단일 워크로드 또는 벤치마크	(Track A,B,C) 실험 목적에 맞는 워크로드 다수 선정 후 단독/동시 실행
자원 분할 효과	분할이 전력에 미치는 영향을 직접 검증하지 않음	(Track B,C) CPU/메모리 비율 변경이 전력에 무영향임을 실증, GPU idle+utilization 성분 분리, 스토리지 fio 캘리브레이션 기반 단위 에너지 계수도출
전력 프로파일링 안정성	동시 실행 시 워크로드별 전력 특성 변화 미검증	(Track B) Solo vs Concurrent에서 프로파일 보존 확인
배포 모드 영향	CPU/GPU 배포 차이의 에너지 영향 미고려	(Track B) 동일 모델의 CPU vs GPU 배포 시 에너지 범주 전환 확인

-주요 비교 대상 연구:

- Ferroni et al. (2017), Hè et al. (2024): 멀티테넌트 서버 인프라의 전력 소비 모델 → 자원 분할의 전력 영향을 직접 실험하지 않음
- Guan et al. (2024, WattScope): 비침입적 전력 분해 → 자원 할당 변화에 따른 행동은 분석하지 않음
- Kakolyris et al. (2024): LLM 추론 에너지 모델 → 단일 워크로드 유형에 한정
- FaaSMeter (Sharma & Fuerst, 2024): 서버리스 에너지 프로파일링 → 자원 할당과 시스템 전력의 관계 미검증

2.3 제안 내용

2.3.1.트랙 A: 컴퓨터 자원 이용률과 에너지 소비 사이의 비선형성

기존의 선형적 에너지 분리가 잘못되었음을 우선 증명한다.

제안 1: 워크로드 특성에 따른 비선형 전력 모델 수립

기존 연구들은 전력과 부하의 관계를 단순한 직선으로 모델링해왔다. 그러나 실제 시스템에서는 부하가 증가할수록 전력이 곡선 형태로 증가하는 경우가 많다. 즉, 일차함수의 단순비례 관계가 아니다. 트랙 A에서는 CPU와 GPU 각각에 대해 선형 모델과 2차 다항 모델을 모두 구성하여 실제 데이터를 더 잘 설명하는 모델을 ~~비교한다. 도출한다.~~ 특히 GPU 중심과 CPU 중심 워크로드 사이에서 이 곡선의 형태가 어떻게 다르게 나타나는지를 ~~분류별로~~ 분석한다.

제안 2: GEMM 교차 실험을 통한 하드웨어 아키텍처 효과 분리

동일한 GEMM 연산을 CPU와 GPU에서 각각 실행하여 전력/부하 곡선을 오버레이한다. 부하 생성 방식을 CPU는 taskset 코어 수 제한, GPU는 batch size 조절로 통제하여 하드웨어 아키텍처 차이만을 독립 변수로 격리한다. 두 곡선의 기울기 차이와 포화점 위치를 비교함으로써 비선형성이 워크로드 알고리즘 속성이 아닌 하드웨어 ~~마이크로~~아키텍처에서 기인하는지 판별한다.

2.3.2. 트랙 B: 다중 워크로드 환경에서의 에너지 소비 분리

~~9종의 AI 및 비AI 워크로드에 대해 50개 실험 구성(단독 10개 + 동시 실행 10조합 * 4배율 = 40개)을 수행하여 다음을 제안한다.~~

다중 워크로드 실행 시 에너지 소비 특성을 분석하여, 정확한 에너지 분리 모델을 제안하기 위한 초석을 마련한다.

제안 1: 자원 할당 둔감성 (Allocation Insensitivity)

워크로드 조합이 고정된 상태에서 CPU 코어와 메모리 비율을 변경해도 전체 시스템 전력의 차이가 ~~유의미하지 않다는 것을 보여줌으로써 할당된 자원의 양이 에너지 소비량에 영향을 주지 않음을 증명한다. ±1.4% 이내로 변하지 않음~~ → 10개 조합 중 9개에서 확인; 예외 1개(Combination H)는 CPU 자원 부족으로 인한 CPU 파이프라인 병목

제안 2: 워크로드 전력 프로파일 보존 (Profile Preservation)

단독 실행과 동시 실행에서 각 워크로드의 CPU 이용률은 $\pm 3\%$, GPU 전력은 $\pm 5W$ 이내로 유지한다는 것을 보여줌으로써 전력 특성은 워크로드 고유의 성질이며, 동반 워크로드에 의해 변하지 않는다는 것을 증명한다. → 전력 특성은 워크로드 고유의 성질이며, 동반 워크로드에 의해 변하지 않음

제안 3: 배포 모드 기반 에너지 범주 전환 (Deployment-Dependent Energy Shift)

동일한 ResNet-18 모델이 CPU 배포 시 108W(CPU-dominant), GPU 배포 시 216W(GPU-dominant)로 다르다는 것을 보여줌으로써 에너지 소비의 특성은 실행 하드웨어 (배포 모드)에 의존적임을 증명한다. → 에너지 범주는 애플리케이션 이름이 아니라 실행 하드웨어(배포 모드)에 의해 결정됨

제안 4: 이용률 기반 에너지 카테고리 프레임워크

GPU-dominant: SM 이용률이 주요 전력 결정 인자 (GEMM 95% → 224W vs LLM-GPU 25% → 57W), CPU-dominant: CPU 이용률이 주요 전력 결정 인자 (ResNet-CPU 46% → 54W vs NodeJS 5% → 27W), 메모리 관련 파라미터(RSS, 프레임버퍼)는 전력 크기에 미미한 영향을 미친다.

2.3.3.트랙 C: 실제 VM 환경으로의 모델 확장 및 검증

컴퓨터 자원별 독립적 귀속 모델을 도출하여 에너지 사용량을 정확하게 분리할 수 있는 모델을 제안한다. 이 모델을 통해 에너지 소비량 기반 과금 시스템을 설계할 수 있다. 해당 연구에서는 실제 과금 모델까지는 제안하지 않지만, 과금 모델을 만드는데 근거가 되는 에너지 귀속 모델을 제안하는 것에 의의가 있다.

제안 1: 시스템 수준 에너지 분해 및 추론 기반 귀속 모델

측정 가능한 유일한 ground truth인 시스템 전체 에너지(E_sys)를 자원 유형별 (CPU·GPU·메모리·스토리지·기타)로 분해하고, 각 자원의 물리적 에너지 행동 특성에 근거한 귀속 규칙을 통해 워크로드별 에너지를 간접 추론한다. 이로써 직접 측정 없이도 워크로드 수준의 에너지 비용을 합리적으로 산출한다.

제안 2: 자원 유형별 독립적 귀속 전략

자원	에너지 특성	귀속 전략
CPU	사용률 비례(utilization-driven), 직접 측정 가능	RAPL 측정 active 에너지를 cgroup CPU 사용률 비율로 배분
GPU	할당 기반 idle 성분 + 활동 기반 active 성분 혼재	idle → vGPU 할당 비율, active → SM 사용률 비율로 이중 분리
메모리(DRAM)	정적 전력 지배(allocation-driven), 워크로드별 측정 수단 부재	할당 용량 비례(0.2 W/GB, LPDDR4 사양 기반)
스토리지(NVMe)	I/O 활동 비례(activity-driven)	fio 캘리브레이션 계수(β_r, β_w) × 워크로드별 I/O 볼륨

제안 3: 에너지 보존 일관성 보장 (3단계 파이프라인)

- Step 1. 시스템 에너지 분해 및 베이스라인 분리: $E_{sys} = E_{cpu} + E_{gpu} + E_{mem} + E_{sto} + E_{other}$; 워크로드 실행 전 idle 상태에서 자원별 베이스라인 에너지를 측정하고 고정 기준값으로 설정
 - E_{sys} : 전체 시스템 전력
 - E_{cpu} : RAPL intel-rapl:0 (package) 에너지
 - E_{gpu} : nvidia-smi power.draw 적분값
 - E_{mem} : RAPL intel-rapl:0:2 (DRAM) 에너지
 - E_{sto} : 스토리지 I/O 관련 에너지 (간접 추정)

- E_{other} : 칩셋, 팬, PSU 손실 등 (벽면 전력 - 컴포넌트 합)
- Step 2. 자원별 워크로드 에너지 귀속: 각 자원의 에너지 행동 특성에 맞는 독립적 귀속 전략을 적용하여 워크로드별 에너지 산출 (2.3.3 제안 2 참고)
- Step 3. 에너지 보존 일관성 보장: $E_{sys} = E_{baseline} + \sum E_{wi} (i = 1, \dots, n)$; 모든 귀속 에너지의 합과 베이스라인의 합이 시스템 실측 에너지와 일치함을 수식으로 보장

2.4 기대 효과 및 의의

- VM 환경 제공 기업에게 에너지 인식 과금 모델의 실증적 근거 제공: 할당 기반 과금이 에너지 비용과 괴리됨을 실험으로 입증하여, 워크로드 이용률 기반 과금으로의 패러다임 전환 근거 마련
- 클라우드 운영 최적화: 워크로드 전력 프로파일이 동시 실행에서도 보존되므로, 단독 측정 결과만으로 Multi-Tenant 환경의 전력을 예측 가능, 즉, 명확한 에너지 소비 모델링을 통해 다수의 이용자가 컴퓨팅 자원을 공유하는 상황으로 확장 가능.
- 학문적 기여: 학문적으로 에너지 인식 스케줄링 및 자원 관리 연구의 실험적 기반 제공

2.5 주요 기능 리스트

2.5.1. 트랙 A: 컴퓨터 자원 이용률과 에너지 소비 사이의 비선형성

기능 1. 실험 전 환경 자동 검증: 실험 재현성을 보장하기 위해 실험 시작 전 시스템 환경을 자동으로 점검한다.

기능 2. 실시간 메트릭 수집: 100ms 샘플링 간격으로 9종 메트릭을 백그라운드 스레드에서 병렬 수집하여 CSV로 저장한다.

기능 3. 실험 오케스트레이션: 단일 실험 사이클의 전 과정을 자동으로 실행하며, 6종 워크로드 × 5회 반복을 순차 처리한다.

기능 4. 데이터 분석 및 시각화: 수집된 CSV를 로드하여 6종 그래프와 4단계 모델링 파이프라인을 실행한다.

-4단계 모델링 파이프라인

Step1. Baseline 설정: 단순 선형 전력 모델을 **baseline**으로 설정한다.

Step2. 비선형 모델 수립: 실험 데이터를 기반으로 워크로드 분류별 2차 다항 회귀 모델을 독립적으로 추정한다.

Step3. 모델 검증: Step2에서 추정한 모델에 대해 평균 제곱근 오차 (RMSE, Root Mean Squared Error), 평균 절대 백분율 오차 (MAPE, Mean Absolute Percentage Error)을 도출해 모델의 일반화 성능을 평가한다.

*평균 제곱근 오차: 오차를 제곱하여 평균내기 때문에 큰 오차에 대해 더 엄격하게 패널티를 줄 수 있다.

**평균 절대 백분율 오차: 실제 대비 오차가 몇 %인지를 보여주는 상대적 지표이다.

Step4.비교 분석: Step1의 **Baseline** 모델과 검증과정을 거친 도출 모델을 비교하여 성능 개선폭을 정량적으로 알아본다.

2.5.2.트랙 B: 다중 워크로드 환경에서의 에너지 소비 분리

기능 1. 9종 워크로드 선정 스위트: GPU-dominant 5종(GEMM, Training, ResNet-GPU, YOLO, LLM-GPU) + CPU-dominant 4종(ResNet-CPU, LLM, ffmpeg, NodeJS)

기능 2. 자원 분할 실험 프레임워크: cgroup v2로 VM-A/VM-B 에뮬레이션; **VM-A와 VM-B에 대해 CPU 코어 수와 메모리를 1:1:2:1:3:1:5:1 비율로 동일 총 용량(20코어·28GB) 유지하며 분할**

기능 3. 전력 분해 분석: 컴포넌트별 전력 분해 및 워크로드 조합별 Wall Power 변동 분석(10개 조합 × 4비율 = 40 실험). 이를 시스템 전력 측정 결과와 비교하여 전력을 각 컴퓨터 자원별 소비 전력으로 분해한다.

***Wall Power:** 서버에 실제로 공급되는 전력을 측정하기 위해 콘센트(벽면 소켓)에 외부 파워미터를 연결하여 측정한 실제 소비 전력이다. RAPL 등 소프트웨어 기반 측정값과 달리 전원 공급 장치(PSU) 변환 손실까지 포함한 시스템 전체의 입력 전력을 의미한다.

기능 4. 에너지 카테고리 분류: GPU-dominant / CPU-dominant 분류 프레임워크; 배포 모드에 따른 에너지 범주 전환 확인

기능 5. 프로파일 보존 검증: 단독 vs 동시 실행에서 CPU 이용률 $\pm 3\%$, GPU 전력 $\pm 5W$ 이내 유지 여부 정량 검증

기능 6. 시각화 및 보고: matplotlib 기반 14종 분석 그래프 자동 생성; summary.csv(50행) 및 7종 분석용 CSV 내보내기

2.5.3.트랙 C: 실제 VM 환경으로의 모델 확장 및 검증

기능1. 시스템 에너지 분해 및 베이스라인 분리: $E_{sys} = E_{cpu} + E_{gpu} + E_{mem} + E_{sto} + E_{other}$; idle 베이스라인 측정(RAPL·nvidia-smi·BIOS DIMM 비활성화) 후 고정 기준값으로 설정

기능2. CPU 에너지 귀속: $E_{wi}^{cpu} = E_W^{cpu} \times (U_i^{cpu} / \sum U_j^{cpu})$; RAPL active 에너지를 cgroup v2 CPU 사용률 비율로 배분

기능3.GPU 에너지 귀속(이중 구조): $E_{wi}^{gpu} = E_{idle}^{gpu} \times a_i + E_W^{gpu_act} \times (U_i^{gpu} / \sum U_j^{gpu})$; 할당 기반 idle 성분(vGPU 비율)과 활동 기반 active 성분(SM 사용률) 분리

기능4. 메모리 에너지 귀속: $E_{wi}^{mem} = E_{idle}^{mem} \times (m_i / M)$; LPDDR4 사양(0.2 W/GB) 기반 할당
용량 비례 모델

기능5. 스토리지 에너지 귀속: $E_{wi}^{sto} = \beta_r \times read_i + \beta_w \times write_i$; fio 캘리브레이션으로 β_r, β_w
산출(NVMe: 2.5 J/GB); cgroup v2 I/O 통계로 워크로드별 I/O 볼륨 수집

기능6. 에너지 보존 검증 및 과금 비교: $E_{sys} = E_{baseline} + \sum E_{wi}$ 조건을 수치로 확인; 동일 자원
할당 조건에서 할당 기반 과금 vs 에너지 기반 과금 차이를 배수($\times$) 단위로 정량화

3. Project-Design

3.1 요구사항 정의

각 트랙별 요구사항에 대해, 추후 목차에서 지칭하기 위해 ID를 부여하였다. 이에 더해 요구사항 제목과
자세한 설명을 표로 시각화하여 기술하였다.

3.1.1.트랙 A: 컴퓨터 자원 이용률과 에너지 소비 사이의 비선형성

track A는 컴퓨팅 자원 할당/사용량과 에너지 소비량 간의 비례 관계를 알아내고, 비례 관계에 유의미하게
영향을 주는 지표를 찾는 것을 목표로 4개의 기능을 구현한다.

ID	요구사항	우선순위	설명
A-R1	실험 전 환경 자동 검증	필수	- 워크로드 스크립트 존재 여부 및 Python 의존성(라이브러리 import, CUDA 런타임) 확인 - CPU/GPU 통제 조건 검증: Intel Turbo Boost·C-State disable 여부, GPU/메모리 클럭 고정, CUDA_VISIBLE_DEVICES 설정 - 계측 도구 가용성 확인: RAPL, sensors 명령 응답 확인; 결과를 항목별로 PASS/FAIL/WARN으로 분류 출력
A-R2	실시간 매트릭 수집	필수	- CPU 전력: RAPL(/sys/class/powercap/intel-rapl) 에너지 카운터 차분으로 순간 전력 산출 - GPU 전력: nvidia-smi --query-gpu=power.draw 명령어를 사용하여 GPU 전력 산출 - CPU Busy / GPU Utilization: /proc/stat 및 nvidia-smi - Memory Bandwidth Utilization, IPC: perf stat 사용
A-R3	실험 오케스트레이션	필수	- 온도 기반 Idle 진입 판정: CPU $\leq 45^{\circ}\text{C}$, GPU $\leq 50^{\circ}\text{C}$ 10초 이상 유지 - CPU 중심 워크로드: taskset 코어 수 제한 + numactl 단일 노드 바인딩으로 부하 제어 - GPU 중심 워크로드: batch size 조절로 GPU SM 점유율 제어 - 실험 사이클: Warm-up(측정 제외) → 부하율별 워크로드 실행 → Cool-down. Warm-up과 Cool-down에서 온도를 일정시간 이상 유지하는 정량적 조건을 만족해야함.

A-R4	데이터 분석 및 시각화	필수	- 그래프 1: CPU Busy – CPU 전력 회귀 분석 - 그래프 2: GPU Utilization – GPU 전력 회귀 분석 - 그래프 3: 워크로드별 전력 구성 비율 - 그래프 4: Memory Boundness와 에너지 소비 간 상관관계 - 그래프 5: 부하율별 에너지 추이 - 그래프 6: 교차 워크로드(GEMM) 비교
------	--------------	----	---

3.1.2.트랙 B: 다중 워크로드 환경에서의 에너지 소비 분리

track B는 복수의 워크로드를 동시 실행 시 에너지 소비량 변화를 알아내는 것을 목표로 6가지 기능을 구현한다.

ID	요구사항	우선순위	설명
B-R1	9종 워크로드 자동 실행	필수	GPU 워크로드 5종 + CPU 워크로드 4종의 단독/동시 실행 스크립트
B-R2	4가지 자원 비율 테스트	필수	1:1, 2:1, 3:1, 5:1 비율로 CPU 코어/메모리를 분할하여 실행
B-R3	컴포넌트별 전력 분해	필수	idle 전력을 baseline으로 active 전력(Δ CPU, Δ GPU, Δ DRAM) 분리
B-R4	에너지 카테고리 분류	필수	GPU-dominant / CPU-dominant 분류 프레임워크
B-R5	다중 계층 동시 전력 측정	필수	CPU(RAPL), GPU(nvidia-smi), DRAM(RAPL), Wall(외부 전력계)을 1초 간격으로 동시 측정
B-R6	그래프 생성	필수	matplotlib 기반 14종 분석 그래프 자동 생성

3.1.3.트랙 C: 실제 VM 환경으로의 모델 확장 및 검증

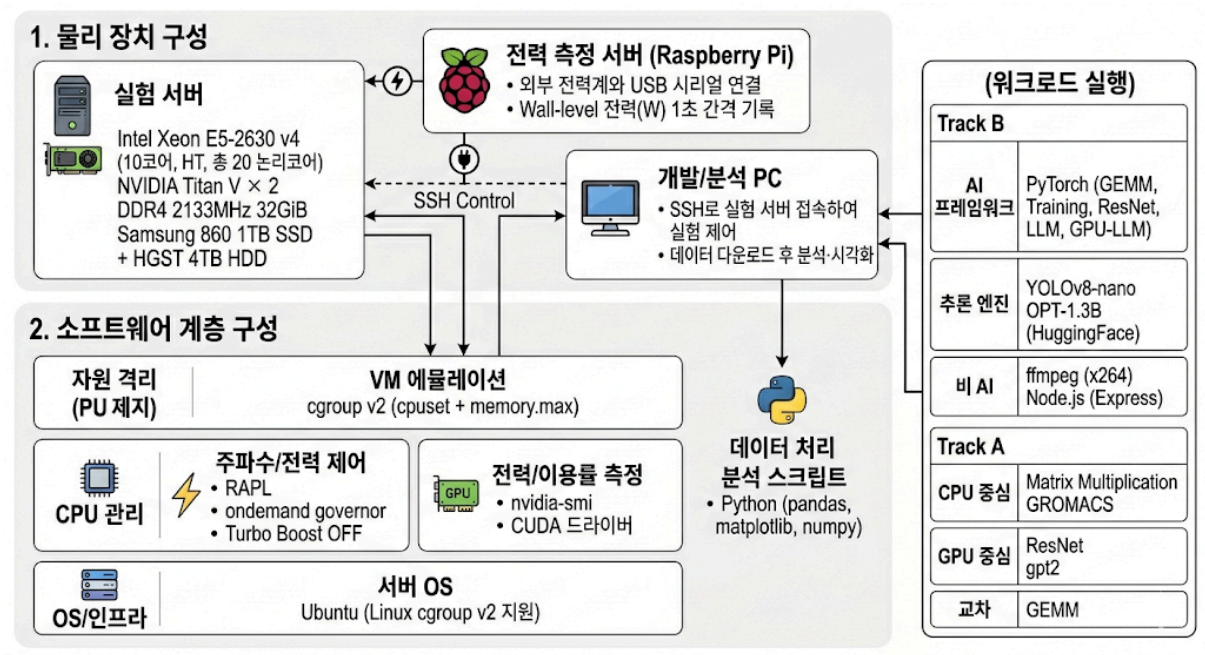
track C는 자원별 귀속 전략과 그에 대한 타당성 및 재현성을 확립하는 것을 목표로 9가지의 기능을 구현한다.

ID	요구사항	우선순위	설명
C-R1	워크로드 수준 에너지 귀속 가능성	필수	공유 자원 환경에서 물리적 분리 측정이 불가능한 조건에서도 워크로드별 에너지 소비를 합리적으로 귀속할 수 있는 추론 기반 귀속 방법론 제시
C-R2	자원 유형별 이질적 에너지 행동 반영	필수	균일 단일규칙이 아닌 자원 유형별 구분된 귀속 전략으로 CPU·GPU·메모리·스토리지의 상이한 에너지 특성 반영
C-R3	AI 워크로드 에너지 특성 규명	필수	현대 AI 추론 워크로드(YOLO, ResNet, GPT2)의 에너지 소비 구조가 전통 워크로드(Node.js)와 어떻게 다른지 자원 구성 수준에서 실증적으로 규명

C-R4	멀티테넌트 환경에서의 귀속 유효성	필수	두 워크로드를 공유 환경에서 동시에 실행했을 때, 동시 실행 공유 환경에서 제안 모델의 귀속 결과가 격리 실행 실측값과 얼마나 일치하는지 정량적으로 검증
G-R5	추가 계측 오버헤드 금지	필수	별도 하드웨어 instrumentation 없이 Intel RAPL, nvidia-smi/NVML, Linux cgroup v2 등 플랫폼 내장 통계만으로 동작
C-R5	에너지 보존 원칙 준수	필수	모든 귀속 에너지의 합과 베이스라인의 합이 시스템 실측 에너지와 수식 수준에서 일치
C-R6	자원 유형별 독립적 귀속 전략	필수	CPU·GPU·메모리·스토리지에 각 자원 특성에 맞는 귀속 방식 분리 적용; GPU는 반드시 할당+활동 이중 구조로 모델링
C-R7	격리 실행 기반 기준값 확보	필수	각 워크로드를 격리 실행하여 idle 성분을 제거한 workload-induced 에너지를 실측(동일 조건 5회 반복 평균)하고 ground truth 로 사용
C-R8	6가지 시나리오 포괄적 검증	필수	AI+전통(3종) 및 AI+AI 대칭·비대칭(3종) 조합을 포함한 총 6가지 동시 실행 시나리오에서 귀속 정확도 검증
C-R9	과금 불형평성 정량화	필수	동일 자원 할당 조건에서 워크로드 간 에너지 차이를 배수(×) 단위로 측정하여 할당 기반 과금의 한계를 실증

3.2 전체 시스템 구성

3.2.1.트랙 A & 트랙 B



물리 장치 구성

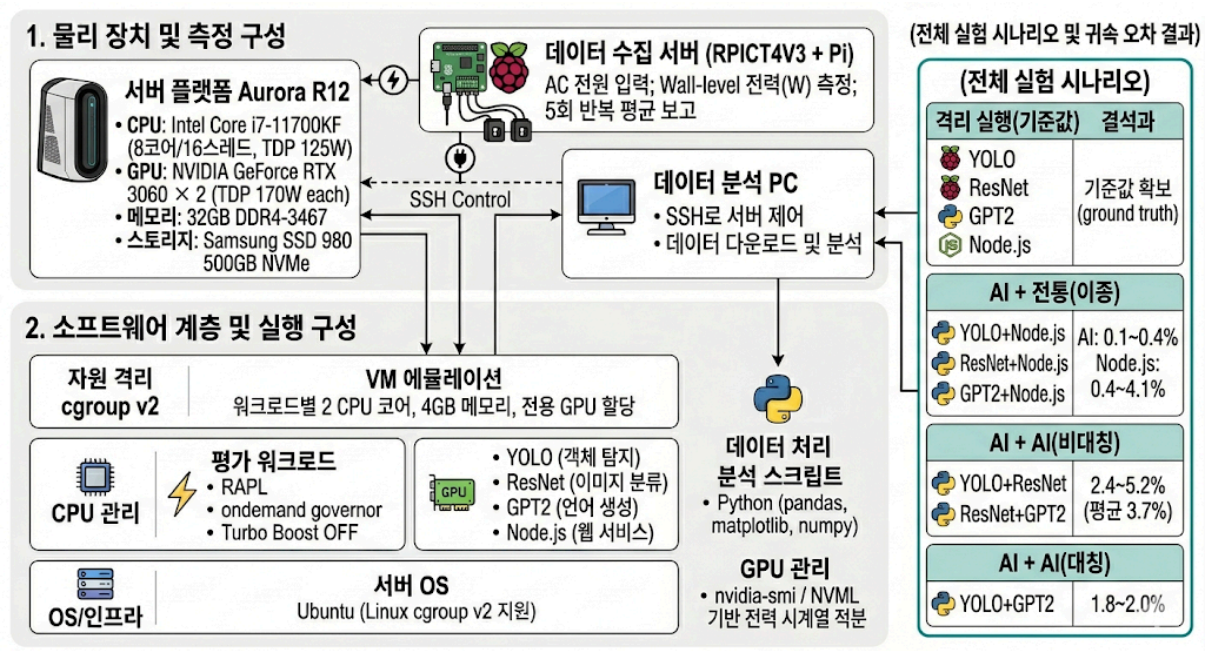
장치	사양
실험 서버	Intel Xeon E5-2630 v4 (10코어, HT, 총 20 논리코어) / NVIDIA Titan V × 2 / DDR4 2133MHz 32GiB / Samsung 860 1TB SSD + HGST 4TB HDD
전력 측정 서버 (Raspberry Pi)	외부 전력계와 USB 시리얼 연결; Wall-level 전력(W) 1초 간격 기록
개발/분석 PC	SSH로 실험 서버 접속하여 실험 제어; 데이터 다운로드 후 분석·시각화

소프트웨어 계층 구성

선행 연구와의 차별점에서 기술하였듯, 실측값을 토대로 한 실증적 분석을 위한 기술을 사용한다.

계층	구성 요소	기술
OS/인프라	서버 OS	Ubuntu (Linux cgroup v2 지원)
자원 격리	VM 에뮬레이션	cgroup v2 (cpuset + memory.max)
GPU 관리	전력/이용률 측정	nvidia-smi, CUDA 드라이버
CPU 관리	주파수/전력 제어	RAPL, ondemand governor, Turbo Boost OFF
워크로드 (트랙 A)	CPU 중심	Matrix Multiplication, GROMACS
	GPU 중심	ResNet, gpt2
	교차	GEMM
워크로드 (트랙 B)	AI 프레임워크	PyTorch (GEMM, Training, ResNet, LLM, GPU-LLM)
	추론 엔진	YOLOv8-nano, OPT-1.3B (HuggingFace)
	비 AI	ffmpeg (x264), Node.js (Express)
데이터 처리	분석 스크립트	Python (pandas, matplotlib, numpy)

3.2.2.트랙 C: 실제 VM 환경으로의 모델 확장 및 검증



공통 실험 환경

지도교수님으로부터 제공받은 서버를 사용하는 Track A,B와는 달리, 실험 목적에 맞게 구축한 서버를 사용한다.

구성 요소	세부 사양
서버 플랫폼	Alienware Aurora R12
CPU	Intel Core i7-11700KF (8코어/16스레드, TDP 125W)
GPU	NVIDIA GeForce RTX 3060 × 2 (TDP 170W each)
메모리	32GB DDR4-3467
스토리지	Samsung SSD 980 500GB NVMe
시스템 에너지 측정	RPiCT4V3 + CT 센서 + Raspberry Pi (AC 입력)
CPU 에너지 측정	Intel RAPL on-chip energy counter
GPU 에너지 측정	NVIDIA nvidia-smi / NVML 전력 시계열 적분
자원 격리	Linux cgroup v2 (워크로드별 2 CPU 코어, 4GB 메모리, 전용 GPU)
평가 워크로드	YOLO(객체 탐지), ResNet(이미지 분류), GPT2(언어 생성), Node.js(웹 서비스)
실험 반복	동일 조건 5회 반복, 평균값 보고

전체 실험 시나리오

시나리오	실행 조합	검증 Feature	귀속 오차 결과
------	-------	------------	----------

격리 실행(기준값)	YOLO/ResNet/GPT2/Node.js 각 단독	기능1~기능5 기준값 확보	(ground truth)
AI + 전통(이종)	YOLO+Node.js, ResNet+Node.js, GPT2+Node.js	기능2, 기능3, 기능6	AI: 0.1~0.4%, Node.js: 0.4~4.1%
AI + AI(비대칭)	YOLO+ResNet, ResNet+GPT2	기능3, 기능6	2.4~5.2%(평균 3.7%)
AI + AI(대칭)	YOLO+GPT2	기능3, 기능6	1.8~2.0%

전체 최대 귀속 오차는 약 5% 이내이며, 할당 기반 과금 대비 에너지 기반 과금의 차이는 AI+Node.js 조합 기준 4.7~11.0배로 측정되었다. 이는 제안 모델이 실용적으로 허용 가능한 오차 범위 내에서 에너지 기반 공정 과금의 실현 가능성을 실증함을 의미한다.

3.3 주요 엔진 및 기능 설계 (상세 설계)

본 프로젝트는 세 개의 트랙이 공통된 실험 인프라(서버, RAPL 인터페이스, nvidia-smi, cgroup v2)를 기반으로 각각 독립적인 연구 파이프라인을 설계 및 구현하였다. 아래에서는 각 트랙의 전체 모듈 설계를 상세하게 기술한다. 모듈 구성은 모듈명, 역할, 주요 기능, 3.1에서 대응되는 feature ID를 기술한 표로 작성하였으며, 각 트랙별 주요하고 특징적인 설계를 설명한다.

3.3.1. 트랙 A: 컴퓨터 자원 이용률과 에너지 소비량 사이의 비선형성 분석

모듈 구성

모듈명	역할	주요 기능	해당 기능
precheck.py	실험 환경 사전 검증	RAPL/NVML/perf 접근 가능 여부 확인, Python/CUDA 의존성 검사, Turbo Boost/C-State 비활성화 여부 확인, PASS/FAIL/WARN 분류 출력	A-R1
collect_metric.py	실시간 메트릭 수집	CPU/GPU 전력, CPU Busy, GPU SM Utilization, DRAM bandwidth, IPC, 온도 데이터를 100ms 간격으로 수집 후 CSV 저장	A-R2
run_experiment.py	실험 오케스트레이션	Idle 안정화, Warm-up 측정, Cool-down 자동 제어, taskset/numactl 기반 CPU 부하 제어, batch size 기반 GPU 부하 제어	A-R3
analyze_data.py	데이터 분석 및 시각화	선형/이차 회귀 분석 (선형은 Baseline으로 사용하기 위함), R² 비교, 비선형성 판정, 6종 그래프 자동 생성	A-R4

workloads/	실험 워크로드 집합	ResNet, GPT-2, GEMM, GROMACS, Matrix Multiplication 등 CPU/GPU/Cross workload 제공	(기능을 실행하기 위한 디렉터리)
-------------------	------------	---	--------------------

핵심 설계 결정

- 온도 기반 **Idle** 상태 판정: 반복 실험 시 이전 **workload** 실행으로 인한 온도 변화가 다음 측정 결과에 영향을 미치는 문제를 방지하기 위해, 단순 **sleep** 기반 대기 대신 온도 기반 **idle stabilization** 방식을 채택하였다.
 - CPU 온도 $\leq 45^{\circ}\text{C}$, GPU 온도 $\leq 50^{\circ}\text{C}$ 조건을 두개 다 만족한 상태에서 10초 이상 유지하여야 한다.
 - 이를 통해 **thermal throttling** (컴퓨터 자원의 온도가 너무 오르면 순간적으로 **frequency** 등의 가동률을 떨어뜨리는 방식) 영향을 최소화한다.
 - leakage current** 증가를 억제한다.
 - 반복 실험 간 초기 상태를 통일한다.
- CPU 부하 수준 제어: CPU workload는 단순 thread 개수 조절만 사용할 경우 OS scheduler migration 및 NUMA remote access가 발생할 수 있으므로, **taskset**과 **numactl**을 함께 사용하는 **locality-aware** 방식으로 설계하였다.
 - Low (4 cores)/Medium (10 cores)/High (20 cores)로 부하수준을 정의하였다.
- GPU 부하 수준 제어: GPU workload는 CPU와 달리 **affinity** (process나 thread를 특정한 CPU 코어에 할당하는 방식. 즉, 각각의 코어에 대해 할일을 분배해주는 것이다)가 어렵기 때문에, **batch size**를 조절하여 GPU SM occupancy를 변화시키는 방식으로 설계하였다.
 - Low (batch size=2)/Medium (batch size=4)/High (batch size=8)로 설정하였다.
- 실시간 메트릭 동기 수집: 메트릭 수집 시, 측정 시점이 다를 경우 **correlation** 분석 오차가 발생할 수 있다. 이를 방지하기 위해 모든 메트릭을 동일 **sampling timestamp** 기준으로 동기 수집하도록 설계하였다.
 - 샘플링 주기: 100ms
 - 수집 대상: CPU package power (RAPL), GPU power (NVML), CPU Busy (/proc/stat), GPU SM utilization, DRAM bandwidth, IPC, CPU/GPU temperature
 - 측정 데이터는 **timestamp** 기반 CSV 형태로 저장하여 이후 회귀 분석 및 에너지 적분 계산에 활용한다.
- 반복 실험 기반 통계 설계: 단일 측정 결과는 **background daemon**, **thermal fluctuation**, **scheduler jitter** 등의 영향을 받을 수 있으므로, 각 **workload x 부하 수준** 조합에 대해 반복 실험을 수행하도록 설계하였다.
 - 반복 횟수: 5회 반복
 - 각 반복 결과에 대해 평균, 표준편차, RMSE, R^2 를 계산하여 통계적 신뢰도를 확보하도록 설계하였다.

실험 프로토콜

환경 검증 → Idle Stabilization (Idle 상태 확인) → 워크로드 실행 → Warm-up (30초) → 측정 (60초) → 워크로드 종료 → Cool-down → 다음 부하 단계 반복

실험 규모

CPU-centric 워크로드: Matrix Multiplication, GROMACS

GPU-centric 워크로드: ResNet, GPT-2

Cross workload: GEMM

부하 수준: CPU (Low/Medium/High), GPU (Low/Medium/High)

전체 실험 수: 6 workloads x 3 load levels x 5 repetitions = 90 times

3.3.2. 트랙 B: 다중 워크로드 환경에서 에너지 사용 분리

모듈 구성

모듈명	역할	주요 기능	해당 기능
run_cgroup_experiments.py	실험 오케스트레이션	cgroup v2 디렉터리 생성, cpuset.cpus/memory.max 설정, 워크로드 프로세스 실행 및 타이밍 제어	B-R1, B-R2
concurrent_power.py	다중 계층 전력 측정	1초 주기로 RAPL CPU/DRAM, nvidia-smi GPU 전력, /proc/stat 시스템 CPU%, /proc/[pid]/stat 프로세스별 메트릭 동시 수집	B-R5
workload_resnet.py	ResNet-18 추론 워크로드	PyTorch ResNet-18, CPU 전용 추론, ImageNet 크기 랜덤 텐서 반복 추론	B-R1
workload_gemm.py	행렬 곱셈 워크로드	PyTorch GEMM (4096×4096), GPU 연산 집약형	
workload_llm.py	LLM 추론 워크로드	GPT-2 텍스트 생성, CPU 모드 (메모리 집약형)	
workload_gpu_llm.py	LLM GPU 추론 워크로드	OPT-1.3B fp16 추론, GPU 모드 (VRAM 집약형)	
workload_training.py	모델 학습 워크로드	ResNet-18 CIFAR-10 학습, GPU 연산 + CPU 데이터 로딩	
workload_yolo.py	YOLO 객체 탐지 워크로드	YOLOv8 Nano 실시간 추론, GPU 경량	
workload_ffmpeg.py	비디오 인코딩 워크로드	FFmpeg H.264 인코딩, CPU 집약형	
server.js + autocannon	웹 서버 워크로드	Node.js Express 서버 + autocannon 부하 생성, 경량 I/O	

wall_power.py	벽면 전력 측정 (Raspberry Pi)	USB 시리얼로 외부 전력 미터 데이터 수집	B-R5
merge_data.py	데이터 병합	서버 CSV와 벽면 전력 CSV를 타임스탬프 기반으로 조인	B-R3
plot_report.py	분석 그래프 생성	Matplotlib 기반 전력 프로파일, 카테고리 분류, 버블 차트 등 생성	B-R4, B-R6
export_gsheet.py	Google Sheets 내보내기	분석 결과를 공유 스프레드시트로 자동 업로드	B-R6

핵심 설계 결정

- **cgroup v2 기반 VM 시뮬레이션:** 실제 VM 환경 대신 Linux cgroup v2를 사용하여 다중 워크로드 환경을 시뮬레이션하는 방식을 채택하였다. 하이퍼바이저 오버헤드 없이 CPU 코어([cpuset.cpus](#))와 메모리([memory.max](#))를 워크로드별로 격리할 수 있으며, [/proc/\[pid\]/stat](#) 기반 프로세스별 메트릭 수집이 가능하다.
 - 총 자원(20코어 + 28GB)을 일정하게 유지하면서 VM-A와 VM-B 간 배분 비율을 1:1, 2:1, 3:1, 5:1의 4단계로 변화시켜, 자원 할당 변화에 따른 에너지 반응을 관측한다.
- **이중 측정 구조 (서버 내부 + 벽면 전력):** 소프트웨어 기반 측정(RAPL, [nvidia-smi](#))만으로는 PSU 손실, 팬, 칩셋 등의 전력이 누락된다. 이를 보완하기 위해 Raspberry Pi에 외부 전력 미터를 연결하여 벽면 전력을 독립적으로 동시 측정하는 이중 구조를 설계하였다.
 - 서버 측: RAPL CPU/DRAM, [nvidia-smi](#) GPU 전력·SM%, [/proc/stat](#) 시스템 CPU%, [/proc/\[pid\]/stat](#) PID별 CPU%, VmRSS, I/O
 - Raspberry Pi 측: USB 시리얼을 통한 벽면 전력(W) 수집
 - 두 데이터를 Unix timestamp 기반 nearest join(± 2.0 초)으로 병합한다.
- **GPU 디바이스 명시적 지정:** 초기 설계에서는 [CUDA_VISIBLE_DEVICES](#) 환경 변수로 GPU를 분리하려 했으나, 두 프로세스를 동시에 [subprocess](#)로 실행할 때 부모 프로세스의 환경 변수가 덮어쓰워지는 문제가 발생하였다. 이를 해결하기 위해 각 워크로드 스크립트 내에서 [--device cuda:N](#) 인자로 직접 GPU를 지정하는 방식으로 변경하였다.
- **1단계 분류 기준 — GPU 전력의 CPU 할당 독립성:** GPU-dominant와 CPU-dominant의 분류 기준으로 "GPU 전력이 CPU 코어 수 변화에 독립적인가"를 사용하였다. 50개 실험에서 GPU-dominant 워크로드(GEMM, ResNet-GPU, YOLO, Training, GPU-LLM)는 CPU 코어를 3개에서 17개로 변경해도 GPU 전력 변동이 $\pm 4\%$ 이내였다. 단, Training은 CPU 파이프라인(DataLoader)에 의존하는 워크로드로, 임계 CPU 자원 이하(10코어 공유 시)에서 SM 사용률이 89%→50%로 급락하는 조건부 독립성(Conditional GPU Independence)을 보였다.
- **2단계 분류 — IPC 기반 CPU-dominant 세분화:** CPU-dominant 워크로드 내에서 동일한 CPU 사용률(~45%)임에도 에너지 소비가 다른 현상(ResNet-CPU 54.4W vs LLM 46.1W)이 관측되었다. 이를 IPC(Instructions Per Cycle) 차이로 설명하며, Compute-bound($IPC > 1.0$:

ResNet-CPU 1.75, ffmpeg 1.32)와 Memory-bound(IPC < 0.5: LLM 0.48)로 세분화하였다.

Phase 2에서 ffmpeg을 추가하여 n=2→3으로 재현성을 검증하였다.

- **Phase 2 워크로드 보강 설계:** Phase 1의 6종 워크로드만으로는 분류 체계의 빈 영역이 존재하였다. 이를 체계적으로 보완하기 위해 3종의 워크로드를 추가하였다.
 - Training (ResNet-18 CIFAR-10 학습): Training vs Inference 에너지 구조 비교 (backward pass의 VRAM 3.6배, DRAM 2.6배 차이 관측)
 - ffmpeg (x264 인코딩): CPU Compute-bound 재현성 확보 (IPC 1.32로 Compute-bound 영역 확인)
 - GPU-LLM (OPT-1.3B fp16 추론): GPU-dominant 내부 SM-heavy(GEMM 224W) vs Memory-heavy(GPU-LLM 57W) 스펙트럼 완성
- **실험 프로토콜 표준화:** 모든 실험 사이클을 [30초 idle 안정화 → 120초 워크로드 실행 → 30초 cooldown]으로 고정하였다. 워크로드 실행 구간(t=30s~150s)의 phase average만 분석에 활용하여 전이 구간의 노이즈를 제거한다.
 - 총 50개 실험: Solo 10회(9종 + idle) + Concurrent 40회(10조합 × 4비율)
 - 보충 실험 12회 (cross-partner 검증 포함 시 총 62회)

실험 프로토콜

하나의 실험 사이클은 다음과 같이 구성된다:

[30초 idle 안정화] → [120초 워크로드 실행] → [30초 cooldown]

↑ (t=30s)

↑ (t=150s)

└─ 전체 120초 phase average 분석 ─┘

- idle 안정화(30초): CPU 온도 및 전력이 기준선으로 복귀할 때까지 대기
- 워크로드 실행(120초): 두 워크로드를 동시에 실행하며, 전체 120초 구간(PHASE_START=30s ~ PHASE_END=150s)의 phase average를 분석에 활용
- cooldown(30초): 다음 실험 전 시스템 상태 복귀

총 실험 규모

- 단독 실행(solo): 9개 워크로드 + 1개 idle = **10회**
- 조합 실행(concurrent): 10개 워크로드 조합 × 4개 자원 비율 = **40회**
- 보충 실험(cross-partner 검증): K, L, D_0302 조합 × 4개 자원 비율 = **12회**
- **총 62회** 실험, 실험당 약 3분 소요

3.3.3.트랙 C: 실제 VM 환경으로의 모델 확장 및 검증

모듈 설계 및 구성

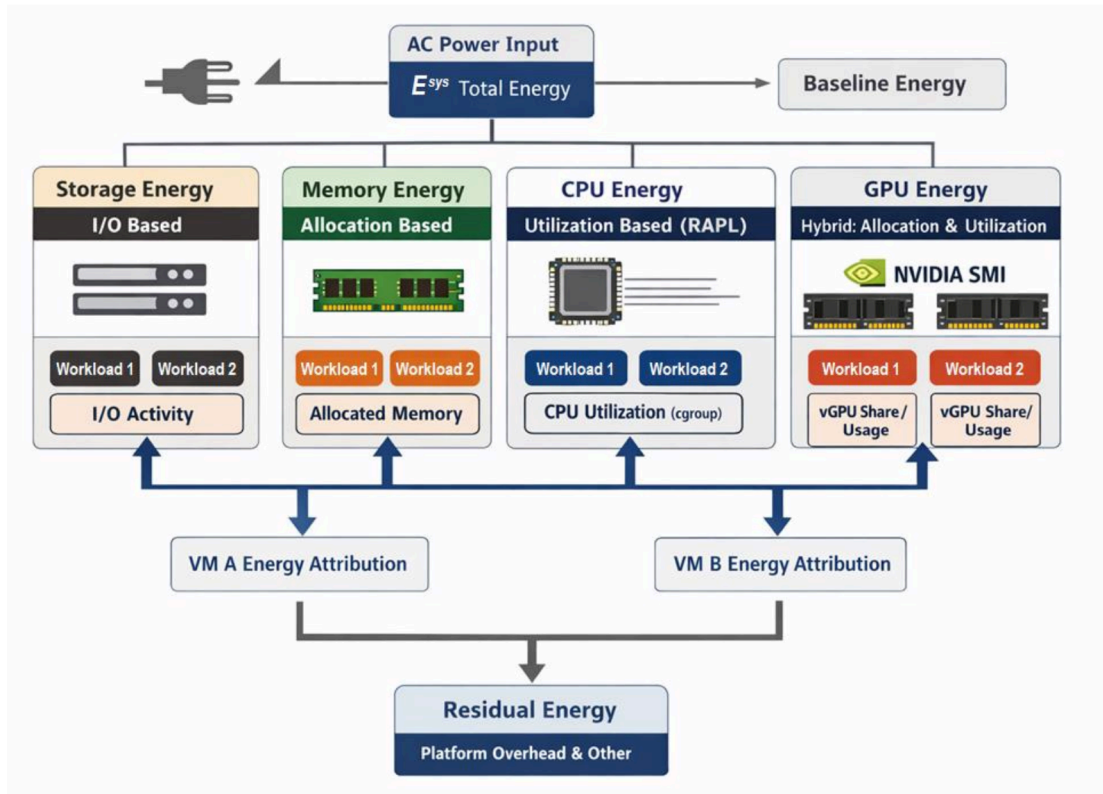


Fig. 1. Overall structure of the proposed energy model across resource components.

모듈명	경로	역할	해당 기능
host_logger.py	scripts/measurement/	RAPL 에너지 카운터 + nvidia-smi GPU 전력을 1초 주기로 기록	C-R2, C-R5, C-R6
cgroup_logger.py	scripts/measurement/	cgroup 기반 워크로드별 CPU%, GPU%, 메모리, I/O 수집	C-R1, C-R2, C-R5, C-R6
rpict_logger.py	scripts/measurement/	RPIC4V3 + CT 센서를 통한 AC 벽면 전력 측정 (독립 검증용)	C-R4, C-R5, C-R7
workload_yolo.py	scripts/workloads/	YOLOv5 객체 탐지 추론	C-R3, C-R7, C-R8, C-R9
resnet18_inference.py	scripts/workloads/	ResNet-18 이미지 분류 추론 (torchvision ResNet-18, batch=32, GPU 배치 추론 반복) (GPU-dominant, SM-heavy)	
gpt2_inference.py	scripts/workloads/	GPT-2 텍스트 생성 (transformers GPT2LMHeadModel, max_length=200, GPU 추론 반복) (GPU-dominant, memory-heavy)	

<i>server.js</i>	<i>scripts/workloads/</i>	Node.js Express 서버 + <i>wrk/autocannon</i> 부하 생성 (<i>server_light.js</i> , <i>server_heavy.js</i> 변형 포함) (Non-AI baseline)	
<i>component_isolation_analysis.py</i>	<i>scripts/analysis/</i>	DIMM 탈착 + GPU 제거 4개 물리 구성(Config A~D)으로 Others 전력을 GPU idle, DIMM, Fixed Others로 완전 분해	
<i>phase2.2_attribution_analysis.py</i>	<i>scripts/analysis/</i>	CPU(사용률 비례)+GPU(이중 구조)+MEM(할당 비례)+STO(I/O 계수) 워크로드별 에너지 귀속 계산	C-R1, C-R2, C-R4, C-R5, C-R6, C-R8, C-R9
<i>energy_attribution_validation.py</i>	<i>scripts/analysis/</i>	$E_{sys}=E_{baseline}+\sum E_{wi}$ 보존 방정식 검증. RPICT 독립 측정값 대비 오차를 산출	C-R4, C-R5, C-R7, C-R8

에너지 모델 수식

시스템 총 에너지는 다음과 같이 분해된다:

$$E_{sys} = E_{cpu} + E_{gpu} + E_{mem} + E_{sto} + E_{other}$$

여기서:

- E_{cpu} : RAPL intel-rapl:0 (package) 에너지
- E_{gpu} : nvidia-smi power.draw 적분값
- E_{mem} : RAPL intel-rapl:0:2 (DRAM) 에너지
- E_{sto} : 스토리지 I/O 관련 에너지 (간접 추정)
- E_{other} : 칩셋, 팬, PSU 손실 등 (벽면 전력 - 컴포넌트 합)

핵심 설계 결정

- **자원별 차등 귀속 전략:** 각 하드웨어 자원의 에너지 소비 메커니즘이 상이하므로, 하나의 통일된 귀속 방식 대신 자원의 물리적 특성에 부합하는 차등 귀속 전략을 설계하였다.
 - **CPU: 사용률 비례 귀속** — RAPL로 측정한 총 CPU 에너지에서 idle을 분리한 후, cgroup CPU 사용률 비율로 분배. CPU는 DVFS가 시스템 수준에서 작동하므로 사용률 기반 분배가 물리적으로 타당하다.
 - **GPU: 할당+사용률 혼합** (이중 구조) — idle 전력은 할당 비율로, active 전력은 SM 사용률 비율로 분배
 - **Memory: 할당 비례 귀속** — DRAM 에너지는 idle/refresh 전력이 지배적이고 activity에 의한 증분이 미미하므로, 할당 용량 비율로 분배
 - **Storage: I/O 활동량 비례** — fio 캘리브레이션으로 산출한 단위 에너지 계수(β_r , β_w)와 워크로드별 read/write 바이트를 곱하여 산출

- **GPU 에너지 이중 구조 (Dual Structure):** GPU는 연산이 없어도 할당만으로 상당한 idle 전력(~15-30W)을 소비한다. 사용률만으로 귀속하면 GPU를 점유만 하는 워크로드의 idle 전력이 누락되고, 할당만으로 귀속하면 실제 연산량 차이가 반영되지 않는다. 이를 해결하기 위해 vGPU 기반 파티셔닝을 가정한 이중 구조를 설계하였다.
 - Idle 컴포넌트: $E^{\text{gpu}}_{\text{base}(w_i)} = E^{\text{gpu}}_{\text{idle}} \times a_i$ (할당 자체에 대한 비용)
 - Active 컴포넌트: $E^{\text{gpu}}_{\text{act}(w_i)} = E^{\text{gpu}}_{\text{act}_W} \times (U^{\text{gpu}}_i / \sum_j U^{\text{gpu}}_j)$ (실제 연산에 비례하는 비용)
- **DRAM 정적 전력 분리 (DIMM 탈착 실험):** DRAM idle 에너지($E^{\text{mem}}_{\text{idle}}$)를 정밀하게 측정하기 위해 물리적 DIMM 탈착 실험을 수행하였다. 전체 메모리(M)와 절반 메모리(M/2) 구성에서 각각 idle 전력을 측정하고, 그 차이로 DRAM 정적 전력을 분리하였다. 실제로는 LPDDR4 데이터시트의 per-capacity 에너지 스펙(0.2 W/GB)을 활용하여 할당량 비례 귀속에 적용하였다.
- **fio 캘리브레이션 기반 스토리지 에너지 계수 산출:** 스토리지 에너지는 소프트웨어적으로 직접 측정할 수 없으므로, fio 도구를 사용한 사전 캘리브레이션 실험으로 단위 에너지 계수를 산출하였다. 블록 크기 1MB, I/O depth 32의 순차 read/write 워크로드를 실행하고, 총 시스템 에너지에서 CPU/GPU/Memory/Others를 제외하여 스토리지 에너지만을 분리하였다.
 - NVMe SSD 기준 스토리지 에너지 계수: 약 2.5 J/GB
 - 워크로드별 스토리지 에너지: $E^{\text{sto}}_{w_i} = \beta_r \times \text{read}_i + \beta_w \times \text{write}_i$
- **RPIC4V3 독립 검증 구조:** 모델 검증 시 자기 참조(self-referencing)를 방지하기 위해, RPIC4V3 + CT(Current Transformer) 센서를 이용한 AC 벽면 전력 독립 측정 체계를 구축하였다. 소프트웨어 모델이 귀속한 에너지 합산값과 독립 측정된 벽면 전력을 비교하여 오차율을 산출한다.
 - 검증 방식: 동시 실행 시 모델 귀속 전력과 단독 실행 시 독립 측정 전력을 비교
 - 결과: AI+Node.js 조합에서 AI 워크로드 오차 0.1~0.4%, AI+AI 조합에서 평균 오차 3.7%, 최대 오차 약 5% 이내
- **에너지 보존 방정식 설계:** 워크로드별 귀속 에너지의 합이 시스템 총 에너지와 일치하는지를 수학적으로 보장하는 보존 방정식을 설계하였다.
 - $E_{\text{sys}} = E_{\text{baseline}} + \sum_i E_{w_i}$
 - Baseline에는 미귀속된 CPU idle, GPU idle의 비할당분, Memory idle의 비할당분, E_{other} 가 포함된다.
 - 이를 통해 모델의 완전성(에너지 누락·중복 계상 없음)을 검증한다.

- **추가 계측 오버헤드 제로 설계:** 논문의 핵심 설계 원칙으로, 모델이 플랫폼에서 기본 제공하는 메트릭(RAPL, nvidia-smi, cgroup 통계)만을 사용하도록 하였다. 별도의 프로파일링 도구나 커스텀 커널 모듈을 요구하지 않아, 실제 엣지 컴퓨팅 환경에 추가 비용 없이 바로 배포 가능하다.

3.3.4. 서버 환경

Track C는 Track A/B와 다른 서버를 사용하여 모델의 일반화 가능성을 검증한다:

항목	Track A/B 서버	Track C 서버
모델	Custom Build (X99-E WS)	Alienware Aurora R12
CPU	Intel Xeon E5-2630 v4 (10C/20T)	Intel i7-11700KF (8C/16T, 125W TDP)
GPU	NVIDIA Titan V × 2	NVIDIA GeForce RTX 3060 (170W TDP) × 2
RAM	32 GB DDR4-2133	32 GB DDR4-3467
Storage	Samsung 860 1TB SSD + HGST 4TB HDD	Samsung SSD 980 500GB NVMe
벽면 전력 측정	Raspberry Pi + USB 전력 미터	RPICT4V3 + CT 센서

3.4 주요 기능의 구현 (상세 구현)

각 트랙의 핵심 기능을 구현 수준에서 상세히 기술한다. 실제 코드 구조, 알고리즘, 데이터 처리 로직을 포함한다.

3.4.1 Track A — 비선형성 분석 구현

Track A는 단일 워크로드 실행 시 자원 이용률(Resource Utilization)과 전력 소비(Power Consumption) 사이의 비선형(non-linear) 관계를 분석하는 것을 목표로 한다.

기능 1. 실험 전 환경 자동 검증 (A-R1, precheck.py)

Track A는 실험 재현성과 측정 신뢰성 확보를 위해 실험 시작 전 `precheck.py`를 실행하여 시스템 상태를 자동 검증한다. 검증 결과는 항목별로 `PASS`, `FAIL`, `WARN`으로 출력되며, 필수 항목 실패 시 실험 수행을 중단한다.

1-1. 워크로드 및 실행 환경 검증

워크로드 스크립트 존재 여부와 Python 의존성 import 가능 여부를 확인한다.

검증 대상:

- `resnet.py`
- `gpt2_infer.py`
- `gromacs_run.py`
- `matmul.py`
- `gemm.py`

특히 `gemm.py`는 CPU/GPU 양 경로 모두 실행 가능한지 검증하기 위해 다음과 같이 테스트 실행을 수행한다.

```
r = run(f"{sys.executable} {GEMM_SCRIPT} --device {device} --size 64")
```

또한 다음 Python 라이브러리의 import 가능 여부를 자동 확인한다.

```
REQUIRED_PYTHON_LIBS = [
    "torch", "torchvision",
    "transformers",
    "numpy", "pandas",
    "psutil"
]
```

PyTorch CUDA 런타임의 정상 동작 여부는 다음 조건으로 확인한다.

```
torch.cuda.is_available()
```

CUDA_VISIBLE_DEVICES 환경 변수도 함께 검증하여 단일 GPU만 사용하도록 강제한다.

1-2. CPU 실험 통제 조건 검증

CPU 전력 측정의 변동성을 최소화하기 위해 DVFS 및 절전 기능 비활성화 여부를 검사한다.

검증 항목:

- `cpufreq governor == performance`
- Intel Turbo Boost 비활성화
- C-State 비활성화
- NUMA 바인딩 가능 여부
- 최소 20코어 사용 가능 여부

Turbo Boost 상태는 다음 인터페이스를 통해 확인한다.

```
/sys/devices/system/cpu/intel_pstate/no_turbo
```

C-State 비활성화 여부는 각 코어의 `cpuidle state` 파일을 검사하여 확인한다.

```
/sys/devices/system/cpu/cpu*/cpuidle/state*/disable
```

실험 실행 시에는 `run_experiment.py`에서 아래와 같이 `governor` 및 `Turbo` 설정을 자동 적용한다.

```
p.write_text("performance")
turbo.write_text("1")
```

또한 모든 `idle state`를 `disable` 처리하여 실험 중 `sleep transition`에 따른 전력 변동을 제거한다.

1-3. GPU 실험 통제 조건 검증

GPU 실험에서는 DVFS 영향을 최소화하기 위해 GPU 및 `memory clock` 고정 여부를 확인한다.

검증 항목:

- GPU 인식 여부
- GPU throttling 상태
- GPU graphics clock 조회 가능 여부
- GPU memory clock 조회 가능 여부
- GPU power limit 조회 가능 여부

실험 시작 시 다음 명령을 사용하여 GPU 클럭 및 TDP를 고정한다.

```
cmds = [
    ["nvidia-smi", "--lock-gpu-clocks=1200,1200"],
    ["nvidia-smi", "--lock-memory-clocks=9251,9251"],
    ["nvidia-smi", "-pl", str(tdp_w)],
]
```

이를 통해 `workload` 간 DVFS 편차를 최소화한다.

1-4. 계측 도구 가용성 검증

실험에 필요한 계측 도구 및 인터페이스의 접근 가능 여부를 사전 확인한다.

검증 대상:

- RAPL 인터페이스
- `perf`
- `sensors`

- `nvidia-smi`
- `/proc/stat`

예를 들어 `perf event` 지원 여부는 다음 명령으로 확인한다.

```
perf stat -e cycles,instructions echo test
```

또한 `cache miss` 이벤트 지원 여부도 별도로 확인한다.

```
perf stat -e cache-misses echo test
```

모든 검증 결과는 아래와 같은 형태로 출력된다.

```
[PASS] torch.cuda.is_available()
[PASS] Intel Turbo Boost 비활성화
[PASS] GPU 클럭 조화 가능
[WARN] C-State 파일 없음
[FAIL] perf 이벤트 cache-misses
```

기능 2. 실시간 메트릭 수집 (A-R2, `collect_metric.py`)

Track A의 메트릭 수집은 `collect_metric.py`의 `MetricCollector` 클래스를 기반으로 구현하였다. 측정은 별도 `background thread`에서 수행되며, 모든 메트릭은 `CSV` 형태로 저장된다. 기본 샘플링 주기는 `100ms`이다.

2-1. CPU 전력 측정

CPU 전력은 Intel RAPL energy counter의 차분값을 이용하여 계산한다.

Track A 구현에서는 `package domain`과 `DRAM domain`을 분리 측정한다.

- `cpu_power_w`: package domain only
- `total_rapl_power_w`: package + DRAM

RAPL energy counter는 다음 경로에서 읽는다.

```
/sys/class/powercap/intel-rapl:*
```

순간 전력은 `energy difference`를 시간 차로 나누어 계산한다.

$$P = \frac{E_{t+\Delta t} - E_t}{\Delta t}$$

Counter wrap-around 가능성을 고려하여 overflow 처리도 구현하였다.

```
if diff < 0:
    diff += 2 ** 32
```

2-2. GPU 전력 및 활용률 측정

GPU 메트릭은 NVML([pynvml](#)) 기반으로 수집한다.

수집 항목:

- GPU power draw
- SM utilization
- memory utilization
- memory usage
- SM clock
- memory clock
- GPU temperature

GPU 전력은 다음 API를 사용하여 읽는다.

```
pynvml.nvmlDeviceGetPowerUsage(h)
```

SM utilization은 NVML sampling API 기반으로 수집하며 fallback으로 utilization rate API를 사용한다.

```
pynvml.nvmlDeviceGetUtilizationRates(h).gpu
```

이를 통해 GPU workload의 부하 수준 변화에 따른 SM occupancy 변화를 측정한다.

2-3. CPU Busy 측정

CPU Busy는 단순 system-wide utilization이 아니라, workload에 할당된 코어 집합만을 기준으로 계산한다. 예를 들어, assigned_cores = "0-3" 인 경우 [_parse_cores\(\)](#)를 통해 [\[0, 1, 2, 3\]](#)으로 변환 후 해당 코어 평균 사용률을 계산한다.

```
valid = [per_cpu[i] for i in self._core_list]
cpu_busy = sum(valid) / len(valid)
```

이를 통해 background process 영향을 최소화하고 실제 workload utilization만 측정하도록 설계하였다.

2-4. DRAM Bandwidth 및 메모리 지표

DRAM bandwidth는 Intel uncore IMC performance counter를 사용하여 측정한다.

구현에서는 다음 perf event를 사용한다.

```
event=0x04,umask=0x0c  
event=0x04,umask=0x30
```

CAS count 기반으로 bandwidth를 계산한다.

```
Bandwidth=\frac{CAS\ Count\times64}{\Delta t}
```

$$Bandwidth = \frac{CAS\ Count \times 64}{\Delta t}$$

또한 memory-boundness 분석을 위해:

- cache misses
- LLC misses
- GPU memory utilization

등도 함께 수집 가능하도록 설계하였다.

2-5. 저장 데이터 형식

수집 데이터는 다음 컬럼 구조로 CSV 저장된다.

```
timestamp, elapsed_s | workload, load_level | repeat | sample_dt_s | cpu_power_w |  
total_rapl_power_w | gpu_power_w | cpu_busy_pct | gpu_util_pct | gpu_mem_util_pct |  
dram_bw_gb_s | cpu_temp_c | gpu_temp_c
```

특히 `sample_dt_s`를 저장하여 실제 샘플 간격 기반 energy 계산이 가능하도록 구현하였다.

기능 3. 실험 오케스트레이션 (A-R3, run_experiment.py)

Track A는 `run_experiment.py`를 중심으로 workload 실행, warm-up, 측정, cooldown을 자동화하였다.

3-1. Idle 안정화 및 Cool-down

실험 간 thermal interference를 제거하기 위해 workload 실행 전 idle stabilization을 수행한다.

조건

- CPU $\leq 45^{\circ}\text{C}$
- GPU $\leq 60^{\circ}\text{C}$
- 최소 idle duration 만족

구현 함수:

- `wait_idle_stable()`
- `wait_for_cooldown()`

온도는 다음 API를 통해 읽는다.

```
read_cpu_temp()  
read_gpu_metrics()["temperature.gpu"]
```

조건이 일정 시간 유지될 때까지 polling 방식으로 대기한다.

3-2. CPU workload 부하 제어

CPU workload는 taskset과 numactl을 이용하여 부하 수준을 조절한다.

부하 수준:

- 20% → 4 cores (0-3)
- 50% → 10 cores (0-9)
- 100% → 20 cores (0-19)

실행 예시는 다음과 같다.

```
taskset -c 0-3 \  
numactl --cpunodebind=0 --membind=0 \  
python matmul.py
```

이를 통해:

- 활성 코어 수 제한
- NUMA remote memory access 제거
- CPU locality 유지

를 동시에 달성한다.

3-3. GPU workload 부하 제어

GPU workload는 batch size를 조절하여 GPU utilization을 변화시킨다.

예시 설정:

```
"batch" : {20: 2, 50: 4, 100: 8}
```

3-4. 실험 사이클

전체 실험 흐름은 다음 순서로 구성된다.

Idle Stabilization → Warm-up (30초) → Measurement (60초) → Cooldown

Warm-up 단계에서는:

- CUDA context initialization
- memory allocation
- JIT overhead 제거

를 수행하며 측정 데이터에는 포함하지 않는다.

Measurement 단계에서는 `MetricCollector`를 활성화하여 100ms 간격으로 메트릭을 수집한다.

출력 파일 형식: results/<workload>/rep<NN>/load<L>.csv

*예시: results/resnet/rep01/load50.csv

기능 4. 데이터 분석 및 시각화 (A-R4, analyze_data.py)

수집된 CSV 데이터는 `analyze_data.py`를 사용하여 분석 및 시각화한다. 분석 단계에서는 workload별 반복 실험 데이터를 평균·표준편차 기반으로 집계하고, 선형 및 이차 회귀 모델을 동시에 적용하여 비선형성을 분석한다.

4-1. 회귀 분석

Track A는 선형 모델과 이차 모델을 동시에 fitting한다.

선형 모델:

$$P = aU + b$$

이차 모델:

$$P = aU^2 + bU + c$$

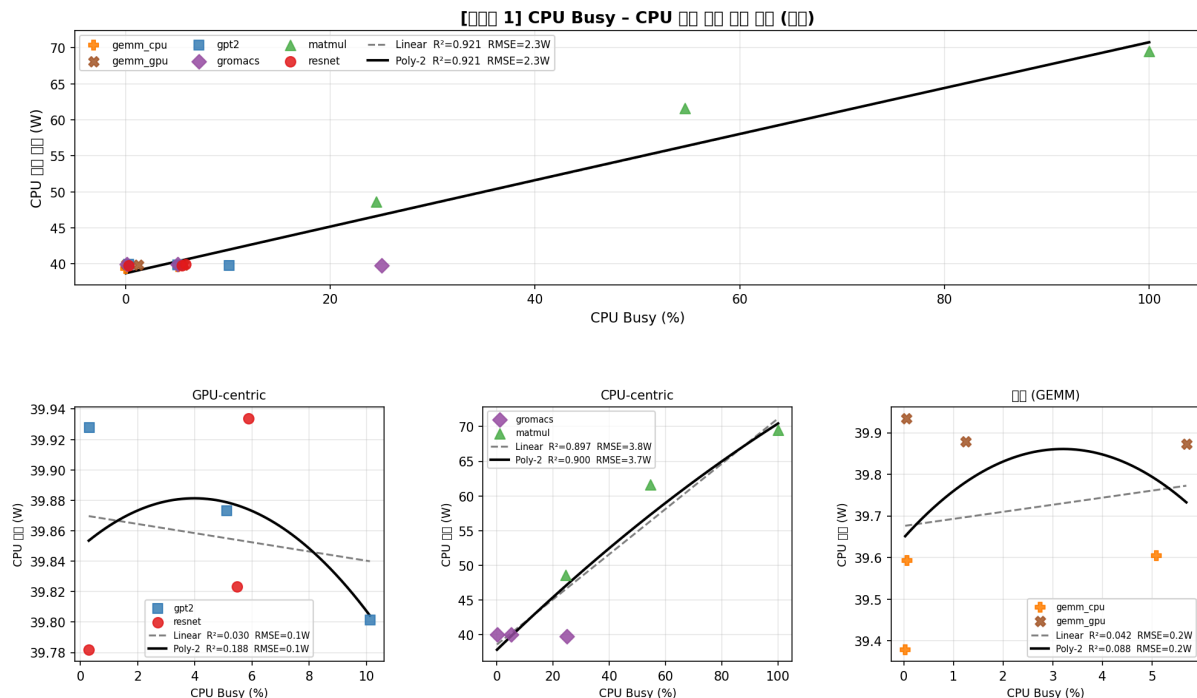
`LinearRegression`과 `PolynomialFeatures(2)` 기반 pipeline을 사용한다.

```
make_pipeline(  
    PolynomialFeatures(2, include_bias=False),  
    LinearRegression()  
)
```

각 모델에 대해 R^2 과 RMSE (Root Mean Square Error) 를 계산하여 모델 적합도를 비교한다.

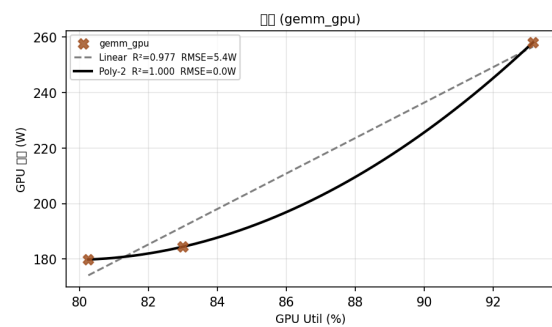
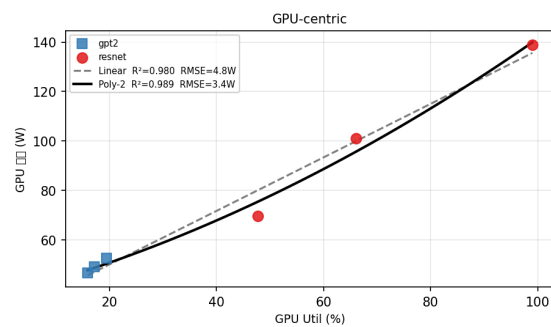
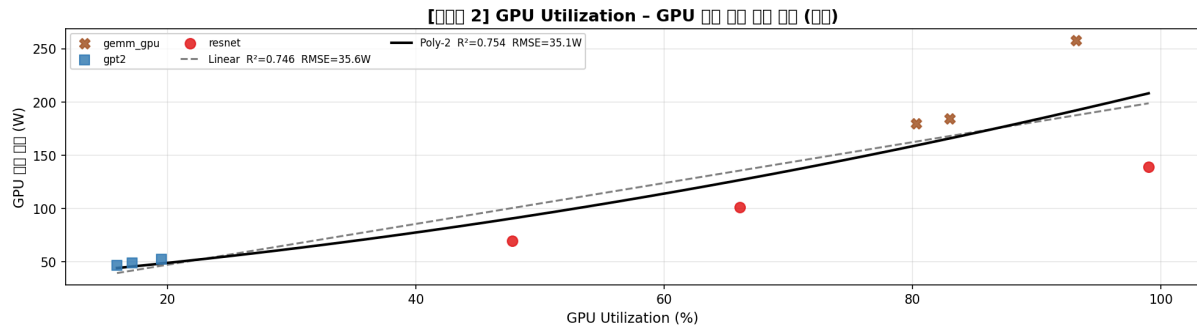
4-2. 그래프 1 — CPU Busy vs CPU Power

CPU utilization과 package power 사이의 관계를 분석한다. scatter plot, linear regression overlay, polynomial regression overlay, R^2 / RMSE annotation를 포함하여 그래프를 작성한다.



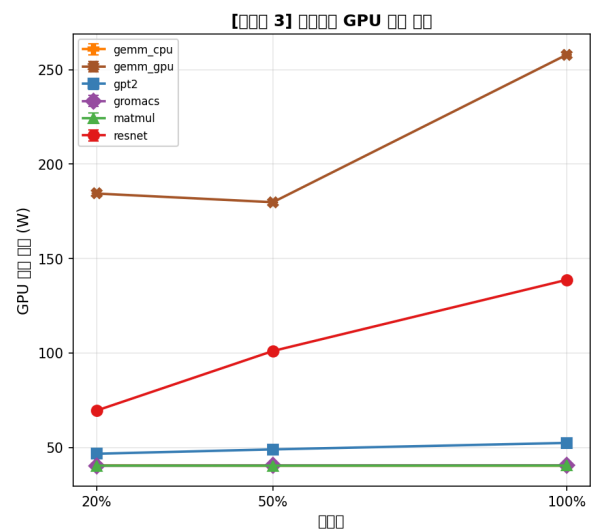
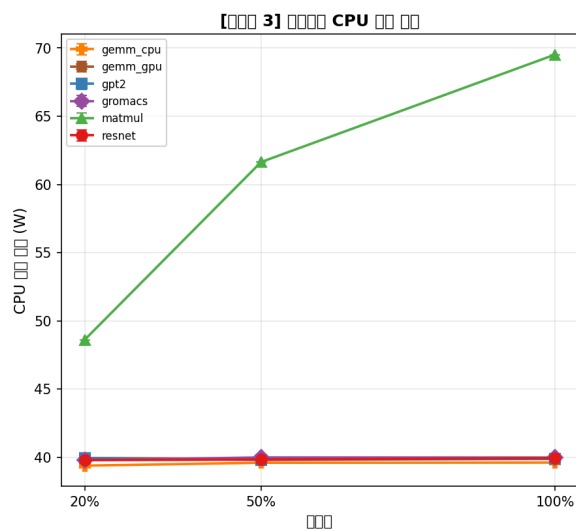
4-3. 그래프 2 — GPU Utilization vs GPU Power

GPU SM utilization과 GPU power draw 관계를 분석한다. GPU-centric workload와 cross workload를 분리하여 시각화한다.



4-4. 그래프 3 — 부하율별 CPU/GPU 전력 추이

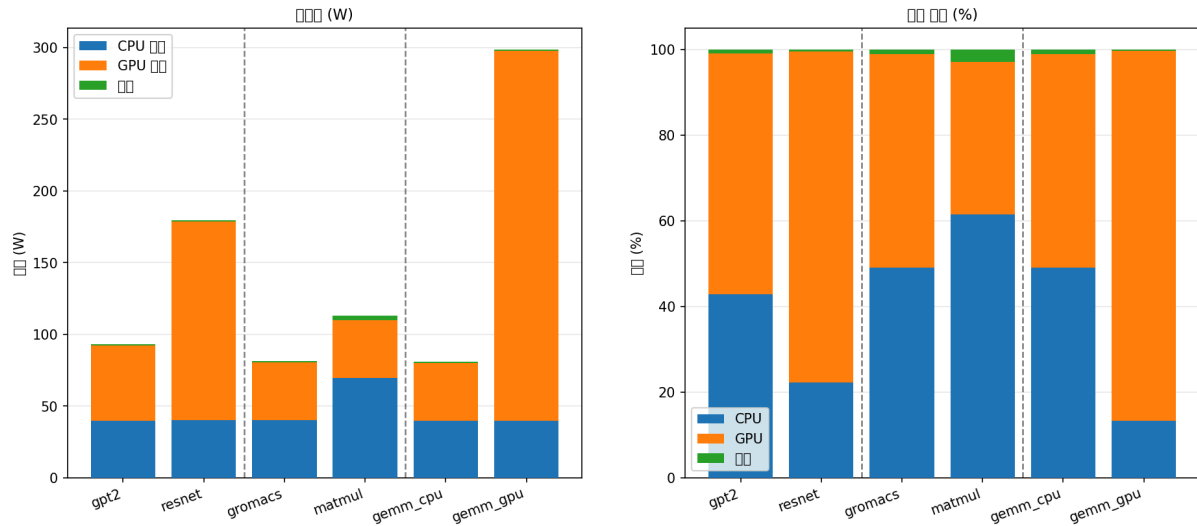
부하 수준 변화에 따른 CPU와 GPU 소비전력량 차이의 변화를 error bar와 함께 시각화한다. X축은 부하율로 20%, 50%, 100%로 표시하고, Y축은 (평균 전력 $\pm$ 표준편차)를 표시한다.



4-5. 그래프 4 — 워크로드별 전력 구성 비율

100% 부하 기준으로 CPU 소비전력, GPU 소비전력과 그 외 전력을 비율을 stacked bar chart로 시각화한다. 이를 통해 compute-bound, memory-bound, CPU-dominant, GPU-dominant 워크로드 간 전력 분포 차이를 비교한다.

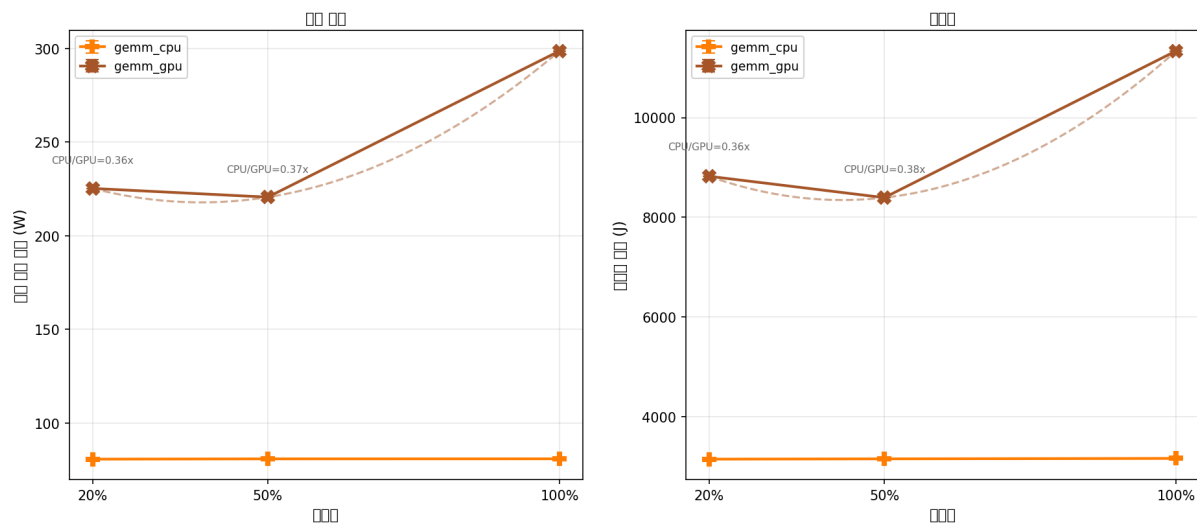
【그래프 4】 워크로드 전력 분포 (100% 부하)



4-6. 그래프 5 — 교차 워크로드(GEMM CPU vs GPU)

동일 GEMM 연산을 CPU에서 실행한 경우와 GPU에서 실행한 경우로 나누어 total power와 total energy를 비교 분석한다. 또한 각 부하 수준에서 CPU/GPU 에너지 비율을 annotation으로 표시한다.

【그래프 5】 GEMM CPU vs GPU 전력 및 에너지

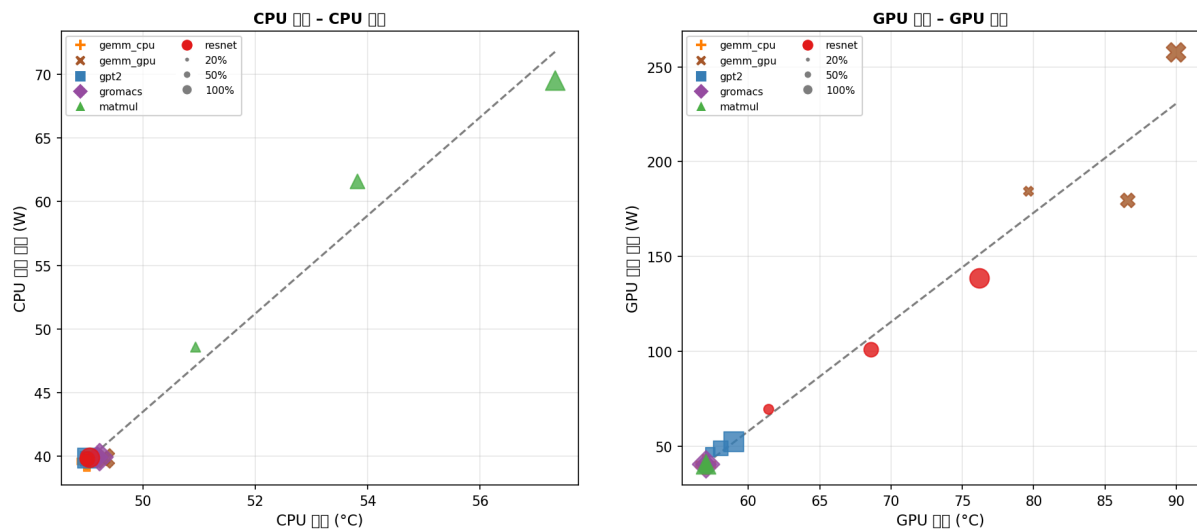


4-7. 그래프 6 — 온도-전력 상관관계

CPU/GPU 온도 변화와 전력 소비 간 상관관계를 분석한다. Workload 별로 다른 모양의 marker를 사용해 표시하고, 부하율 (20/50/100)은 marker의 크기로 표시한다. 이를 통해 thermal throttling 및 leakage

증가 경향을 분석한다.

【표 6】 CPU - GPU



3.4.2. 트랙 B: 다중 워크로드 에너지 분리 구현

기능 1: 자원 분할 실험 프레임워크 (B-R1, B-R2/run_cgroup_experiments.py)

1-1. cgroup v2 기반 자원 분할

Linux cgroup v2를 이용하여 두 개의 워크로드(VM-A, VM-B에 대응)에 CPU 코어, 메모리, GPU를 독립적으로 할당한다.

cgroup 생성 및 자원 제한 설정:

```
# cgroup v2 디렉터리 생성 (optimus 하위 그룹)
os.makedirs("/sys/fs/cgroup/optimus/vm_a", exist_ok=True)
os.makedirs("/sys/fs/cgroup/optimus/vm_b", exist_ok=True)

# cpuset.cpus 설정 -- VM별 CPU 코어 할당
write("/sys/fs/cgroup/optimus/vm_a/cpuset.cpus", "0-9")      # VM-A: 코어 0~9
write("/sys/fs/cgroup/optimus/vm_b/cpuset.cpus", "10-19")    # VM-B: 코어 10~19

# memory.max 설정 -- VM별 최대 메모리 제한
write("/sys/fs/cgroup/optimus/vm_a/memory.max", "14G")       # VM-A: 14GB
write("/sys/fs/cgroup/optimus/vm_b/memory.max", "14G")       # VM-B: 14GB

# 워크로드 프로세스를 cgroup에 등록
write("/sys/fs/cgroup/optimus/vm_a/cgroup.procs", str(pid_a))
write("/sys/fs/cgroup/optimus/vm_b/cgroup.procs", str(pid_b))
```

1-2. 4가지 자원 할당 비율

총 자원(20코어 + 28GB)을 일정하게 유지하면서 VM-A와 VM-B 사이의 배분 비율을 4단계로 변화시킨다:

비율 (A:B)	VM-A CPU 코어	VM-B CPU 코어	VM-A 메모리	VM-B 메모리
1:1	10	10	14 GB	14 GB
2:1	14	6	20 GB	8 GB
3:1	15	5	24 GB	4 GB
5:1	17	3	24 GB	4 GB

1-3. GPU 할당 및 버그 수정

초기 설계에서는 CUDA_VISIBLE_DEVICES 환경 변수로 GPU를 분리하려 했으나, 두 프로세스를 동시에 실행할 때 환경 변수가 올바르게 격리되지 않는 문제가 발생하였다.

문제:

```
# 의도: VM-A는 GPU 0, VM-B는 GPU 1만 사용
os.environ["CUDA_VISIBLE_DEVICES"] = "0" # VM-A 프로세스
os.environ["CUDA_VISIBLE_DEVICES"] = "1" # VM-B 프로세스 -- 부모 프로세스
환경이 덮어쓰워짐
```

CUDA_VISIBLE_DEVICES는 프로세스 수준 환경 변수이므로, run_cgroup_experiments.py가 두 워크로드를 subprocess로 실행할 때 부모 프로세스의 환경 변수가 마지막 설정값으로 덮어쓰워지는 문제가 있었다. subprocess.Popen(env=...)으로 개별 환경을 전달하는 방법도 시도했으나, 일부 PyTorch 버전에서 CUDA 컨텍스트 초기화 순서 문제가 발생하였다.

해결) 명시적 디바이스 지정:

```
# 워크로드 스크립트 내에서 직접 디바이스 지정
# VM-A 워크로드: python workload_resnet.py --device cuda:0
# VM-B 워크로드: python workload_gemm.py --device cuda:1

parser.add_argument("--device", type=str, default="cuda:0")
args = parser.parse_args()
device = torch.device(args.device)
model = model.to(device)
input_tensor = input_tensor.to(device)
```

이 방식은 각 워크로드 프로세스가 PyTorch 내부에서 명시적으로 특정 GPU 디바이스를 선택하므로, 환경 변수 격리 문제가 발생하지 않는다.

1-4. PID 추적 메커니즘

`concurrent_power.py`가 프로세스별 메트릭을 수집하려면 각 워크로드의 PID를 정확히 식별해야 한다.
워크로드 스크립트 파일명을 기반으로 `/proc/`을 탐색한다.

패턴 매칭 규칙:

```
# "workload_resnet.py"로 검색하면 workload_resnet_gpu.py도 매칭되는 문제
# → ".py" 확장자를 포함한 정확한 파일명으로 매칭

def find_pid_by_pattern(pattern, exclude_pid=None):
    """pattern: 예) 'workload_resnet.py'"""
    for pid_dir in os.listdir("/proc"):
        if not pid_dir.isdigit():
            continue
        if int(pid_dir) == exclude_pid:
            continue
        try:
            cmdline = open(f"/proc/{pid_dir}/cmdline", "rb").read()
            cmdline = cmdline.replace(b'\x00', b' ').decode('utf-8',
errors='replace')
            if 'concurrent_power' in cmdline:
                continue
            if pattern in cmdline: # 부분 문자열 매칭
                return int(pid_dir)
        except (FileNotFoundError, PermissionError):
            continue
    return None
```

핵심은 `workload_resnet.py`를 검색할 때 `workload_resnet_gpu.py`가 매칭되지 않도록 `.py` 확장자를 포함한 전체 파일명을 패턴으로 사용하여 부분 문자열 매칭(substring match)하는 것이다. 또한 `concurrent_power.py` 자기 자신이 매칭되지 않도록 제외 처리한다.

1-5. 실험 사이클 자동화

```
def run_experiment(combo_name, workload_a, workload_b, ratio):
    # 1. cgroup 설정
    setup_cgroups(ratio)

    # 2. idle 안정화 (30초)
    log("Phase: idle stabilization")
    time.sleep(30)

    # 3. 전력 측정 시작
    power_proc = start_concurrent_power(combo_name, pids=[])

    # 4. 워크로드 동시 실행
    proc_a = subprocess.Popen(["python", workload_a, "--device", "cuda:0"])
    proc_b = subprocess.Popen(["python", workload_b, "--device", "cuda:1"])
```

```

# PID를 power monitor에 등록
register_pids(power_proc, [proc_a.pid, proc_b.pid])

# 5. 120초 실행
time.sleep(120)

# 6. 종료 및 cooldown
proc_a.terminate()
proc_b.terminate()
stop_concurrent_power(power_proc)
time.sleep(30) # cooldown

# 7. cgroup 정리
cleanup_cgroups()

```

기능 2: 다중 계층 동시 전력 측정 (B-R3, B-R5, B-R6/concurrent_power.py, wall_power.py, merge_data.py)

2-1. 측정 아키텍처 개요

concurrent_power.py는 1초 주기로 시스템 수준과 프로세스 수준의 메트릭을 동시에 수집하는 핵심 계측 모듈이다.

```

concurrent_power.py (메인 루프: 1초 주기)
├── 시스템 수준 메트릭
│   ├── RAPL CPU 패키지 전력 (energy_uj 차분)
│   ├── RAPL DRAM 전력 (energy_uj 차분)
│   ├── nvidia-smi GPU 전력 (GPU 0, GPU 1 각각)
│   ├── nvidia-smi GPU 사용률 (SM% GPU 0, GPU 1)
│   └── /proc/stat 시스템 CPU 사용률
├── 프로세스 수준 메트릭 (워크로드 A, B 각각)
│   ├── /proc/[pid]/stat → CPU 사용률 (utime + stime 차분)
│   ├── /proc/[pid]/status → 메모리 사용량 (VmRSS, MiB)
│   ├── /proc/[pid]/io → I/O 속도 (read_bytes, write_bytes → MB/s)
│   └── nvidia-smi pmon → GPU 사용률 (SM%), GPU 메모리 (framebuffer MiB)
└── 벽면 전력 (별도 Raspberry Pi에서 수집 → 나중에 merge)

```

2-2. 시스템 수준 메트릭 수집 상세

RAPL CPU/DRAM 전력:

```
def read_rapl_power(prev_energy, prev_time):
    curr_energy =
int(open("/sys/class/powercap/intel-rapl/intel-rapl:0/energy_uj").read())
    curr_time = time.time()

    delta_energy = curr_energy - prev_energy
    delta_time = curr_time - prev_time
    power_watts = delta_energy / (delta_time * 1e6) #  $\mu\text{J} \rightarrow \text{J}$ ,  $\text{s} \rightarrow \text{W}$ 

    return power_watts, curr_energy, curr_time
```

DRAM 전력도 동일한 방법으로 intel-rapl:0:2 경로에서 읽는다.

GPU 전력 및 사용률:

GPU 메트릭은 전력과 사용률을 별도의 nvidia-smi 호출로 수집한다:

```
def read_gpu_utilization():
    """GPU SM 사용률 수집"""
    result = subprocess.run(
        ["nvidia-smi", "--query-gpu=utilization.gpu",
         "--format=csv,noheader,nounits"],
        capture_output=True, text=True
    )
    return [float(x.strip()) for x in result.stdout.strip().split("\n")]

def read_gpu_power():
    """GPU 전력 수집"""
    result = subprocess.run(
        ["nvidia-smi", "--query-gpu=power.draw",
         "--format=csv,noheader,nounits"],
        capture_output=True, text=True
    )
    return [float(x.strip()) for x in result.stdout.strip().split("\n")]
```

2-3. 프로세스 수준 메트릭 수집 상세

프로세스 CPU 사용률 (/proc/[pid]/task/*/stat — 모든 스레드 합산):

```
def read_proc_cpu(pid, prev_utime, prev_stime, prev_time):
    # /proc/[pid]/task/*/stat에서 모든 스레드의 utime+stime 합산
    task_dir = f'/proc/{pid}/task'
    total_utime, total_stime = 0, 0
    for tid in os.listdir(task_dir):
        try:
            stat = open(f'{task_dir}/{tid}/stat').read().split()
            total_utime += int(stat[13]) # user mode ticks
            total_stime += int(stat[14]) # kernel mode ticks
```

```

        except (FileNotFoundError, IndexError):
            continue
    curr_time = time.time()

    delta_ticks = (total_utime - prev_utime) + (total_stime - prev_stime)
    delta_time = curr_time - prev_time
    clock_ticks_per_sec = os.sysconf("SC_CLK_TCK") # 보통 100

    cpu_pct = (delta_ticks / clock_ticks_per_sec) / delta_time * 100

    return cpu_pct, total_utime, total_stime, curr_time

```

프로세스 메모리 (/proc/[pid]/status):

```

def read_proc_memory(pid):
    for line in open(f"/proc/{pid}/status"):
        if line.startswith("VmRSS:"):
            rss_kb = int(line.split()[1])
            return rss_kb / 1024 # KiB → MiB
    return 0.0

```

프로세스 I/O (/proc/[pid]/io):

```

def read_proc_io(pid, prev_read, prev_write, prev_time):
    io_data = {}
    for line in open(f"/proc/{pid}/io"):
        key, val = line.strip().split(": ")
        io_data[key] = int(val)

    curr_time = time.time()
    delta_time = curr_time - prev_time
    read_mbps = (io_data["read_bytes"] - prev_read) / delta_time / 1e6
    write_mbps = (io_data["write_bytes"] - prev_write) / delta_time / 1e6

    return read_mbps, write_mbps, io_data["read_bytes"],
    io_data["write_bytes"], curr_time

```

프로세스별 GPU 사용률 (nvidia-smi pmon — 백그라운드 지속 모드):

pmon은 일회성(one-shot)이 아닌, 백그라운드에서 지속적으로 실행되는 방식을 사용한다. stdbuf -oL로 라인 버퍼링을 강제하고, 1초 주기로 SM 사용률과 메모리를 수집한다.

```

class GpuPmon:
    """nvidia-smi pmon을 백그라운드로 실행하여 PID별 GPU 메트릭 수집"""
    def start(self):
        self.proc = subprocess.Popen(
            ["stdbuf", "-oL", "nvidia-smi", "pmon", "-d", "1", "-s", "um"],
            stdout=subprocess.PIPE, stderr=subprocess.DEVNULL, text=True
        )
        self._latest = {}
        # 별도 스레드에서 stdout 파싱
        self._thread = threading.Thread(target=self._reader, daemon=True)
        self._thread.start()

    def _reader(self):
        for line in self.proc.stdout:
            if line.startswith("#"):
                continue
            parts = line.split()
            gpu_idx, pid = int(parts[0]), int(parts[1])
            sm_pct = 0 if parts[3] == "-" else int(parts[3])
            mem_pct = 0 if parts[4] == "-" else int(parts[4])
            self._latest[pid] = {"gpu_idx": gpu_idx, "sm_pct": sm_pct,
                                "mem_pct": mem_pct}

    def get(self, pid):
        return self._latest.get(pid, {"sm_pct": 0, "mem_pct": 0})

```

2-4. 벽면 전력 측정 (Raspberry Pi)

벽면 전력은 서버와 독립된 Raspberry Pi에서 측정한다. 외부 전력 미터가 USB 시리얼로 Raspberry Pi에 연결되어 있으며, wall_power.py가 1초 주기로 W 값을 CSV에 기록한다.

```

# wall_power.py (Raspberry Pi에서 실행)
import serial, time, csv

ser = serial.Serial("/dev/ttyUSB0", 9600)
writer = csv.writer(open("wall_power.csv", "w"))
writer.writerow(["timestamp", "wall_power_w"])

while True:
    line = ser.readline().decode().strip()
    power_w = parse_power(line) # 전력 미터 프로토콜에 따라 파싱
    writer.writerow([time.time(), power_w])
    time.sleep(1)

```

2-5. 데이터 병합 (merge_data.py)

서버 CSV(시스템/프로세스 메트릭)와 Raspberry Pi CSV(벽면 전력)를 타임스탬프 기반으로 병합한다.

```
import pandas as pd

server_df = pd.read_csv("server_metrics.csv")
wall_df = pd.read_csv("wall_power.csv")

# 타임스탬프를 기준으로 nearest join (±2.0초 이내)
merged = pd.merge_asof(
    server_df.sort_values("timestamp"),
    wall_df.sort_values("timestamp"),
    on="timestamp",
    tolerance=2.0,
    direction="nearest"
)
```

최종 출력: 62행(10 solo + 40 concurrent + 12 cross-partner 보충)의 summary.csv와 7개의 분석용 CSV 파일 생성.

기능 3: 에너지 카테고리 분류 프레임워크(B-R4, B-R6/plot_report.py)

Track B 논문의 핵심 기여 중 하나는 AI 워크로드를 에너지 소비 특성에 따라 체계적으로 분류하는 프레임워크이다.

3-1. 1차 축: GPU-dominant vs CPU-dominant 분류

```
if GPU_SM_utilization > 0%:
    category = "GPU-dominant"
else:
    category = "CPU-dominant"
```

이 분류는 논문 Figure 6에서 시각화되며, 모든 워크로드가 두 그룹 중 하나로 명확히 분리된다.

3-2. GPU-dominant 내부 스펙트럼 (논문 Figure 7)

GPU-dominant 워크로드를 SM 사용률과 프레임버퍼 메모리 사용량의 2차원 공간에서 세분화한다. 버블 크기는 GPU 전력을 나타낸다.

	SM%	프레임버퍼 (MiB)	GPU 전력 (W)	하위 분류
GEMM	95%	중간	224 W	SM-heavy
ResNet-GPU	83%	중간	163 W	SM-heavy
Training	89%	중간	151 W	SM-heavy
LLM-GPU	25%	2,600 MiB	57 W	Memory-heavy

YOLO	15%	411 MiB	44 W	Lightweight
------	-----	---------	------	-------------

핵심 패턴:

- SM-heavy 그룹(GEMM, ResNet-GPU, Training): SM 사용률이 높을수록 GPU 전력도 높음 → SM 사용률이 GPU 전력의 주요 결정 요인
- Memory-heavy 그룹(LLM-GPU): SM 사용률은 낮지만 프레임버퍼 사용량이 매우 높음 → VRAM 접근 패턴이 에너지 특성을 결정
- Lightweight 그룹(YOLO): SM과 메모리 모두 낮음 → GPU 자원을 경량으로 사용

3-3. CPU-dominant 내부 스펙트럼 (논문 Figure 8)

CPU-dominant 워크로드를 CPU 사용률과 RSS(Resident Set Size) 메모리의 2차원 공간에서 세분화한다. 버블 크기는 시스템 전력을 나타낸다.

워크로드	CPU%	RSS (MiB)	IPC	시스템 전력 (W)	하위 분류
ResNet-CPU	46%	중간	1.75	54 W (induced)	Compute-bound
ffmpeg	34%	중간	1.32	49 W (induced)	Compute-bound
LLM	45%	1,220 MiB	0.48	46 W (induced)	Memory-bound
NodeJS	5%	87 MiB	-	27 W (induced)	Lightweight

핵심 인사이트:

- ResNet-CPU와 LLM은 CPU 사용률이 유사하지만(46% vs 45%), IPC가 크게 다름(1.75 vs 0.48)
- IPC가 높은 ResNet-CPU는 연산 집약형(compute-bound), IPC가 낮은 LLM은 메모리 집약형(memory-bound)
- 그러나 전력 관점에서는 **사용률(CPU% 또는 SM%)**이 전력 크기를 결정하며, 메모리 파라미터(RSS, 프레임버퍼)는 제한적 영향만 미침

3-4. 분류 프레임워크의 의의

이 에너지 카테고리 분류는 실용적으로 다음을 가능하게 한다:

1. 새로운 워크로드의 에너지 프로파일을 소수의 메트릭(SM%, CPU%, IPC, RSS)만으로 예측
2. 멀티테넌트 환경에서 에너지 효율적인 워크로드 배치(placement) 결정
3. VM별 에너지 과금 모델의 기초 데이터 제공

기능 4: 프로파일 보존 검증 (B-R3, B-R5/plot_report.py, concurrent_power.py)

Track B의 핵심 주장 중 하나는 워크로드의 에너지 프로파일이 동시 실행 환경에서도 보존된다는 것이다. 이를 정량적으로 검증하기 위해 단독 실행(solo)과 동시 실행(concurrent)의 메트릭을 비교하는 검증 로직을 구현하였다.

4-1. Solo vs Concurrent 메트릭 비교

summary.csv에는 solo 10회와 concurrent 40회의 phase average 메트릭이 저장되어 있다. 각 워크로드에 대해 solo baseline과 concurrent 실행 시의 CPU 사용률 및 GPU 전력을 비교한다.

```
def verify_profile_preservation(summary_df):
    """ summary_df: solo + concurrent 실험 결과가 포함된 DataFrame 컬럼:
    experiment_type, workload, cpu_pct, gpu_power_w, ... """

    solo = summary_df[summary_df["experiment_type"] == "solo"]
    concurrent = summary_df[summary_df["experiment_type"] == "concurrent"]
    results = []
    for wl_name in solo["workload"].unique():
        solo_cpu = solo[solo["workload"] == wl_name]["cpu_pct"].mean()
        solo_gpu = solo[solo["workload"] == wl_name]["gpu_power_w"].mean()
        conc_rows = concurrent[ (concurrent["workload_a"] == wl_name) |
        (concurrent["workload_b"] == wl_name) ]
        for _, row in conc_rows.iterrows():
            if row["workload_a"] == wl_name:
                conc_cpu, conc_gpu = row["vm_a_cpu_pct"], row["vm_a_gpu_power_w"]
            else: conc_cpu, conc_gpu = row["vm_b_cpu_pct"], row["vm_b_gpu_power_w"]
            delta_cpu = abs(conc_cpu - solo_cpu) delta_gpu = abs(conc_gpu - solo_gpu)
            results.append({ "workload": wl_name, "combo": row["combo_name"],
            "delta_cpu_pp": delta_cpu, "cpu_pass":
            delta_cpu <= 3.0, "delta_gpu_w": delta_gpu, "gpu_pass": delta_gpu <= 5.0, })
    return pd.DataFrame(results)
```

4-2. 판정 기준

프로파일 보존 여부는 두 가지 기준으로 판정한다:

- CPU 사용률 보존: 단독 실행 대비 동시 실행에서의 CPU 사용률 차이가 $\pm 3\%$ 이내
- GPU 전력 보존: 단독 실행 대비 동시 실행에서의 GPU 전력 차이가 $\pm 5W$ 이내

이 기준값은 실험 과정에서 관측된 측정 노이즈 범위를 고려하여 설정하였다. 1초 주기 샘플링에서 background daemon과 scheduler jitter에 의한 자연 변동을 감안하면, CPU $\pm 3\%$ 와 GPU $\pm 5W$ 는 유의미한 변화와 측정 노이즈를 구분하는 실용적 임계값이다.

4-3. 검증 결과 요약

20개 워크로드 인스턴스(10조합 × 2워크로드) 중 19개가 두 기준을 모두 충족하였다. 유일한 예외는 조합 H(Training + ffmpeg)에서의 Training 워크로드이다. CPU 코어가 10개 공유 상태에서 ffmpeg에 5코어만 할당되자, Training의 DataLoader가 CPU 병목을 겪어 GPU 파이프라인에 idle 시간이 증가하였고, SM 사용률이 89%→50%로 급락하였다. 이 현상은 조건부 GPU 독립성(Conditional GPU Independence)으로 별도 분류하였으며, 임계 CPU 자원 이상에서는 프로파일 보존이 성립함을 확인하였다.

3.4.3. 트랙 C: VM 환경 에너지 귀속 모델 구현

구현 내용을 핵심 컴퓨팅자원인 CPU와 GPU에 대한 주요결과를 영두에 두고 작성하였다. 그렇기 때문에 앞서 기술한 기능 설명과 혼란을 줄 수 있어 대응되는 기능 아이디와 모듈명을 추가로 적었다. 모듈명에서 세부적인 워크로드 모듈명 (resnet18_inference.py 등)은 공통되기에 생략하고, 에너지 귀속 모델링 중심의 모듈명만 기재하였다.

전체적인 흐름은 다음과 같다:

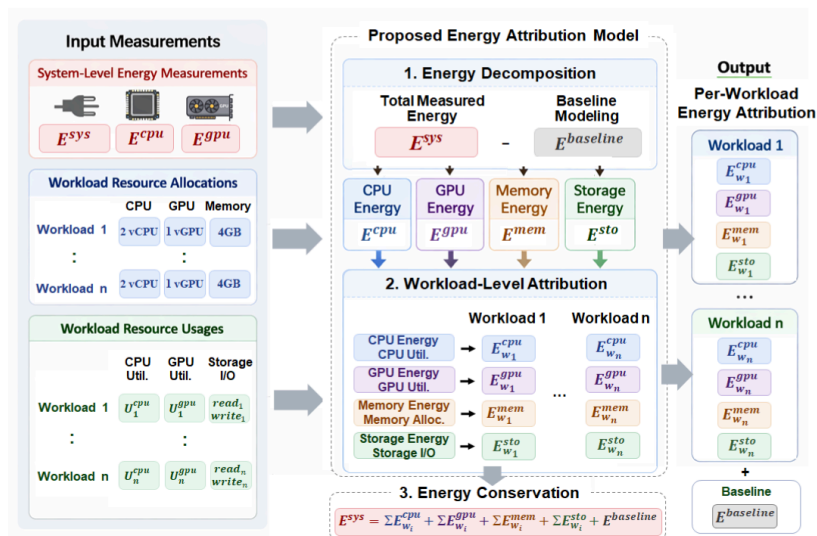


Fig. 2. Structural overview of the proposed energy attribution model, illustrating system-level measurements, resource-wise decomposition, workload-level attribution, and energy conservation.

기능 1: CPU 에너지 귀속 구현 (C-R1, C-R2, C-R3, C-R4, C-R6, C-R7, C-R8/ host_logger.py, cgroup_logger.py, phase2.2_attribution_analysis.py)

1-1. RAPL 에너지 측정과 idle 분리

시스템 전체 CPU 에너지에서 idle(기준선) 에너지를 분리하여 워크로드가 유발한 순수 에너지를 계산한다.

$$E_W^{cpu} = E_{total}^{cpu} - E_{idle}^{cpu}$$

여기서:

- E_{cpu_total} : 워크로드 실행 중 RAPL에서 측정한 총 CPU 에너지
- E_{cpu_idle} : 동일 시간 동안의 idle 상태 에너지 (사전에 idle 실험에서 측정)
- E_W^{cpu} : 워크로드가 유발한 active CPU 에너지

1-2. 프로세스별 CPU 사용률 수집

구현 (host_logger.py + phase2.2_attribution_analysis.py):

```
# cgroup_logger.py 핵심 로직
def collect_per_workload_cpu(pids, interval=1.0):
    prev = {}
    for pid in pids:
        stat = read_proc_stat(pid)
        prev[pid] = {"utime": stat.utime, "stime": stat.stime, "time":
time.time()}

    time.sleep(interval)

    results = {}
    for pid in pids:
        stat = read_proc_stat(pid)
        curr_time = time.time()
        delta_ticks = (stat.utime - prev[pid]["utime"]) + (stat.stime -
prev[pid]["stime"])
        delta_time = curr_time - prev[pid]["time"]
        cpu_pct = (delta_ticks / os.sysconf("SC_CLK_TCK")) / delta_time *
100
        results[pid] = cpu_pct
    return results
```

1-3. 귀속 공식

다중 워크로드 환경에서 각 워크로드 i의 CPU 에너지 귀속:

$$E_{w_i}^{cpu} = E_W^{cpu} \times (U_i^{cpu} / \sum_j U_j^{cpu})$$

여기서:

- U_i^{cpu} : 워크로드 i의 평균 CPU 사용률 (%)

- $\sum_j U_j^{cpu}$: 모든 동시 실행 워크로드의 CPU 사용률 합
- E_W^{cpu} : 시스템 전체 active CPU 에너지

예시: YOLO(CPU 6%)와 Node.js(CPU 20%)가 동시 실행될 때:

$$E_{cpu_yolo}^{cpu} = E_W^{cpu} \times (6 / (6 + 20)) = E_W^{cpu} \times 0.231$$

$$E_{cpu_nodejs}^{cpu} = E_W^{cpu} \times (20 / (6 + 20)) = E_W^{cpu} \times 0.769$$

기능 2: GPU 에너지 귀속 (이중 구조) 구현 (C-R1, C-R2, C-R3, C-R4, C-R6, C-R7, C-R8/host_logger.py, phase2.2_attribution_analysis.py)

GPU 에너지 귀속은 CPU와 달리 이중 구조(dual structure)를 사용한다. GPU는 아무 연산이 없어도 할당(allocation)만으로 상당한 idle 전력을 소비하기 때문이다.

2-1. 이중 구조의 필요성

GPU idle 전력: ~15-30W (할당만 해도 소비)
GPU active 전력: ~50-250W (연산에 비례)

만약 active 에너지만 귀속하면, GPU를 할당받았지만 가볍게 사용하는 워크로드(예: YOLO, SM 15%)의 idle 전력이 누구에게도 귀속되지 않는 문제가 발생한다.

2-2. Idle 컴포넌트 (할당 기반)

$$E_{base_{w_i}}^{gpu} = E_{idle}^{gpu} \times a_i$$

여기서:

- $E_{gpu_idle}^{gpu}$: GPU idle 상태의 에너지 (idle 실험에서 측정)
- a_i : 워크로드 i의 GPU 할당 비율 (예: 2개 워크로드가 공유하면 각각 0.5)

이 컴포넌트는 GPU를 할당받은 것 자체에 대한 에너지 비용이다.

2-3. Active 컴포넌트 (사용량 기반)

$$E_{act_{w_i}}^{gpu} = E_{act_W}^{gpu} \times (U_i^{gpu} / \sum_j U_j^{gpu})$$

여기서:

- $E_{gpu_act_W}^{gpu} = E_{gpu_total}^{gpu} - E_{gpu_idle}^{gpu}$: 워크로드가 유발한 active GPU 에너지
- U_i^{gpu} : nvidia-smi pmon에서 측정한 워크로드 i의 SM 사용률

이 컴포넌트는 실제 GPU 연산에 비례하는 에너지 비용이다.

2-4. 총 GPU 에너지 귀속

$$E_{w_i}^{gpu} = E_{base_{w_i}}^{gpu} + E_{act_{w_i}}^{gpu} = E_{idle}^{gpu} \times a_i + E_{act_w}^{gpu} \times (U_i^{gpu} / \sum_j U_j^{gpu})$$

구현 (host_logger.py + phase2.2_attribution_analysis.py):

```
def attribute_gpu_energy(gpu_total_j, gpu_idle_j, workload_sm_pcts,
                        allocation_fracs):
    """
    gpu_total_j: 실험 중 총 GPU 에너지 (J)
    gpu_idle_j: idle 상태 GPU 에너지 (J, 동일 시간 기준)
    workload_sm_pcts: {wl_name: avg_sm_pct} 워크로드별 평균 SM%
    allocation_fracs: {wl_name: fraction} 워크로드별 GPU 할당 비율
    """

    gpu_active_j = gpu_total_j - gpu_idle_j
    total_sm = sum(workload_sm_pcts.values())

    attribution = {}
    for wl_name in workload_sm_pcts:
        # idle 컴포넌트 (할당 기반)
        base = gpu_idle_j * allocation_fracs[wl_name]
        # active 컴포넌트 (사용량 기반)
        if total_sm > 0:
            active = gpu_active_j * (workload_sm_pcts[wl_name] / total_sm)
        else:
            active = 0
        attribution[wl_name] = base + active

    return attribution
```

2-5. 디바이스 바인딩

Track C에서는 각 워크로드를 전용 GPU 디바이스에 바인딩하여 깨끗한 측정을 보장한다:

```
# resnet18_inference.py (실제 코드 기반)
device = torch.device("cuda:0") model =
models.resnet18(weights=models.ResNet18_Weights.DEFAULT) model =
model.to(device) # GPU 0에 바인딩 model.eval()

# gpt2_inference.py (동시 실행 시)
device = torch.device("cuda:0") model =
```

```
GPT2LMHeadModel.from_pretrained("gpt2") model = model.to(device) # 동일 GPU
공유 시 pmon으로 프로세스별 구분 model.eval()
```

기능 3: 에너지 보존 검증 및 과금 비교 구현 (C-R4, C-R5, C-R6, C-R7,
C-R9/energy_attribution_validation.py, phase2.2_attribution_analysis.py, rpict_logger.py)

3-1. 보존 방정식

에너지 귀속 모델의 적합성을 검증하기 위해, 전체 시스템 에너지가 기준선과 개별 워크로드 귀속의 합으로 보존되는지 확인한다.

$$E_{sys} = E_{baseline} + \sum_i E_{w_i}$$

여기서:

- E_{sys} : RPICT4V3 + CT 센서로 독립 측정한 시스템 총 에너지
- $E_{baseline}$: 워크로드에 귀속되지 않은 기준선 에너지
- $\sum_i E_{w_i}$: 모든 워크로드에 귀속된 에너지의 합

3-2. 기준선 에너지 상세

$$E_{baseline} = E_{idle}^{cpu} + E_{idle}^{gpu} \left(1 - \sum_i a_i\right) + E_{idle}^{mem} \left(1 - \sum_i m_i/M\right) + E^{other}$$

여기서:

- E^{cpu_idle} : CPU idle 에너지
- $E^{gpu_idle} \times (1 - \sum_i a_i)$: 어떤 워크로드에도 할당되지 않은 GPU idle 에너지
- $E^{mem_idle} \times (1 - \sum_i m_i/M)$: 워크로드가 사용하지 않는 메모리의 idle 에너지
- E^{other} : 칩셋, 팬, PSU 손실 등 귀속 불가 에너지

3-3. 검증 구현 (energy_attribution_validation.py)

```
def validate_conservation(measured_total_w, attributed_powers, baseline_w,
tolerance=0.05):
    """
    measured_total_w: 벽면 전력 미터로 측정한 총 전력 (W)
    attributed_powers: {wl_name: power_w} 워크로드별 귀속 전력
    baseline_w: 기준선 전력 (W)
```

```

tolerance: 허용 오차 (기본 5%)
"""

model_total = baseline_w + sum(attributed_powers.values())
error_pct = abs(model_total - measured_total_w) / measured_total_w * 100

result = {
    "measured_total_w": measured_total_w,
    "model_total_w": model_total,
    "baseline_w": baseline_w,
    "attributed": attributed_powers,
    "error_pct": error_pct,
    "pass": error_pct < tolerance * 100
}
return result

```

3-4. 과금 불형평성 분석

동일한 자원 할당(CPU 코어 수, 메모리 용량)을 받은 VM이라도 워크로드 특성에 따라 실제 에너지 소비량은 크게 다를 수 있다. 이 차이를 정량화한다.

```

def compute_pricing_inequity(scenario_results):
    """
    scenario_results: [{workload, allocated_resources, attributed_energy_j},
    ...]
    """

    # 동일 자원 할당을 받은 워크로드 쌍 비교
    for pair in itertools.combinations(scenario_results, 2):
        wl_a, wl_b = pair

        if wl_a["allocated_resources"] == wl_b["allocated_resources"]:

            energy_ratio = (
                max(
                    wl_a["attributed_energy_j"],
                    wl_b["attributed_energy_j"]
                )
                /
                min(
                    wl_a["attributed_energy_j"],
                    wl_b["attributed_energy_j"]
                )
            )

            print(
                f"{wl_a['workload']} vs {wl_b['workload']}: "
                f"{energy_ratio:.1f}x energy difference"
            )

```

```

    )

    print("    → Allocation-based pricing: identical cost")

    print(
        f"    → Energy-based pricing: "
        f"{energy_ratio:.1f}x cost difference"
    )

```

핵심 결과: AI 워크로드 + Node.js 조합에서 동일 자원 할당 하에 에너지 차이가 4.7~11.0배에 달하며, 할당 기반 과금은 이러한 차이를 전혀 반영하지 못한다.

3.5 실험 결과

3.5.1. 트랙 A: 비선형성 분석 실험 계획 및 예비 결과

Track A는 1차 발표 시점에 1차 실험을 완료하였으며, 6종 워크로드에 대해 총 6,913개의 데이터 포인트를 수집하여 3개의 핵심 결과를 도출하였다. 아래에 실험 설계 및 결과를 기술한다. 3.4.1의 기능4에서 유의미한 결과를 추출하여 그래프를 재생성하였다.

실험 설계

항목	내용
워크로드 수	6종 (gemm_cpu, gemm_gpu, gpt2, gromacs, matmul, resnet)
부하 수준	3단계: 20%(4코어), 50%(10코어), 100%(20코어)
반복 횟수	각 조합 1회 (1차 실험 기준)
총 실험 수	6 워크로드 × 3 부하 × ~385 샘플 = 6,913개 데이터 포인트

워크로드 구성 및 데이터 수집 현황:

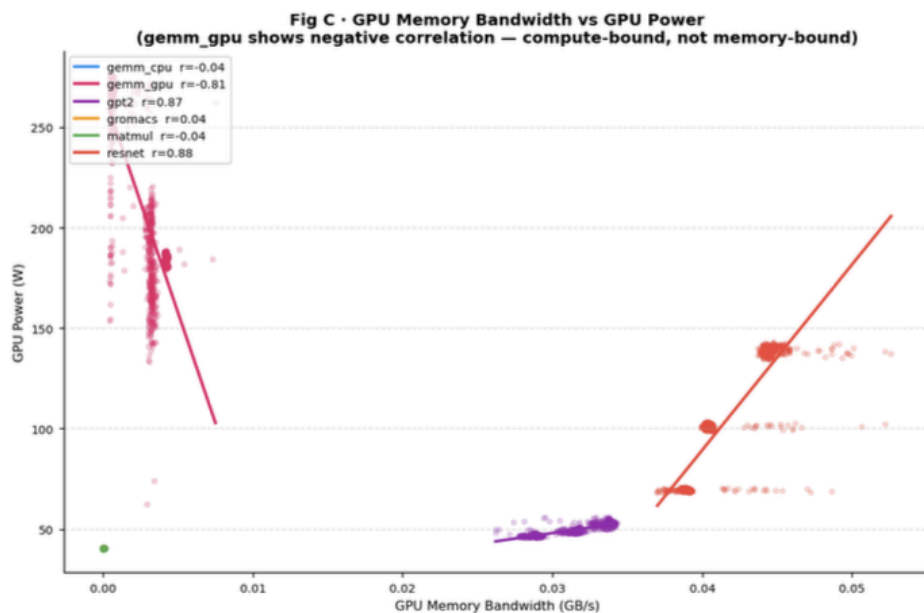
워크로드	유형	카테고리	부하 파라미터	Load 20%	Load 50%	Load 100%
gemm_cpu	CPU	Cross	threads: 4/10/20, size 4096 고정	386개	386개	387개
gemm_gpu	GPU	Cross	batch: 2/4/8 → size 2048/4096/8192	387개	376개	375개
gpt2	GPU	GPU-centric	batch: 2/4/8	388개	386개	385개
gromacs	CPU	CPU-centric	threads: 4/10/20	388개	389개	388개
matmul	CPU	CPU-centric	threads: 4/10/20	387개	378개	378개
resnet	GPU	GPU-centric	batch: 2/4/8	387개	385개	377개

워크로드 × 부하별 평균 총 전력 (CPU RAPL + GPU, W)

워크로드	Load 20%	Load 50%	Load 100%	증가율(20→100%)
gemm_cpu	80.7 W	80.8 W	80.9 W	+0.2%
gemm_gpu	225.3 W	220.6 W	298.8 W	+32.6%
gpt2	87.5 W	89.7 W	93.2 W	+6.5%
gromacs	81.1 W	81.4 W	81.4 W	+0.4%
matmul	90.8 W	104.5 W	113.2 W	+24.7%
resnet	110.2 W	141.8 W	179.6 W	+62.9%

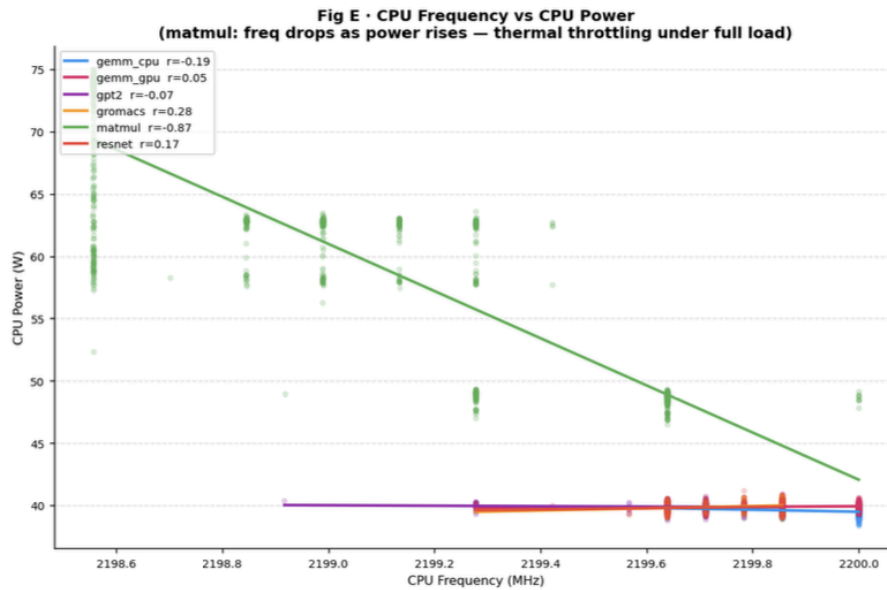
핵심 결과 1: GPU 메모리 대역폭 → GPU 전력

gpt2($r=0.87$), resnet($r=0.88$)은 GPU 메모리 대역폭과 양의 상관을 보여 연산·메모리 접근이 함께 증가하는 memory-compute 혼합형 특성을 나타낸다. 반면 gemm_gpu($r=-0.81$)는 음의 상관으로, SM 연산에 병목이 있는 순수 compute-bound 특성을 보인다.



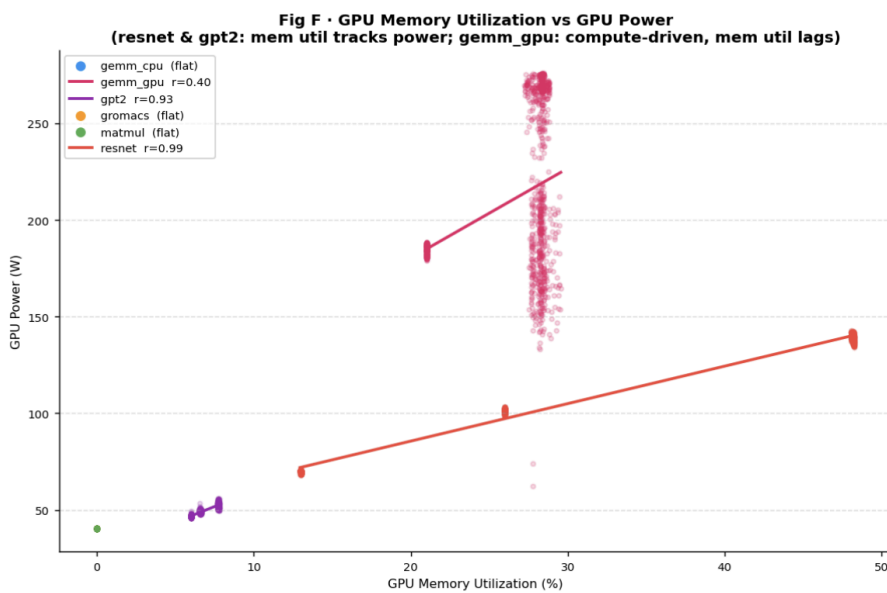
핵심 결과 2: CPU 주파수 → CPU 전력

matmul($r=-0.87$)에서 CPU 주파수와 전력 사이 음의 상관관계가 관찰되었다. 풀 부하 시 서멀 스로틀링(thermal throttling)으로 주파수가 하락하면서 전력이 상승하는 패턴이다. 나머지 워크로드는 Turbo Boost OFF 상태에서 ~2200MHz에 고정.



핵심 결과 3: GPU 메모리 사용률 → GPU 전력

resnet($r=0.99$), gpt2($r=0.93$)은 메모리 사용률이 연산 강도와 직결되는 memory-compute 혼합형이다.
gemm_gpu($r=0.40$)는 SM 연산이 메모리 사용률보다 전력 결정에 더 직접적인 순수 compute-bound이다.



워크로드 유형별 종합 특성

워크로드	전력 지배	부하 민감도	주요 특성
resnet	GPU	매우 높음	GPU Util·Temp·MemBW 모두 전력과 강한 양의 상관. 딥러닝 추론의 전형

gemm_gpu	GPU	중간	Compute-bound. MemBW가 전력과 음의 상관관계 특징
gpt2	GPU	낮음	GPU 사용률 ~17%로 낮지만 Util→Power 상관관계 강함. LM 추론 특성
matmul	CPU	높음	CPU 100% 점유 유일. 서멀 스로틀링 관찰. CPU Busy↔전력 선형 비례
gromacs	CPU	없음	전력 부하 무감각. cpu_busy_pct 이상치 존재
gemm_cpu	CPU	없음	전력 부하 무감각. RAPL 고정 ~40W

track A의 경우 다수의 예비실험과 1회 실험 (위에 기술된 결과)을 수행하였으며, 컴퓨팅 자원 사용량과 에너지 사용량이 단순 비례하지 않는다는 것을 도출하였다. 하지만 논문으로 제안하고자 하는 핵심 주장이 명확하지 않아, 논문 작성 및 투고에는 어려움이 있다.

3.5.2. 트랙 B: Applied Sciences 논문 실험 결과

Track B의 실험 결과는 MDPI Applied Sciences에 게재된 논문의 핵심 발견을 요약한다.

핵심 결과 1: 자원 할당 변화에 대한 시스템 전력 둔감성

10개 워크로드 조합에 대해 4가지 자원 할당 비율(1:1, 2:1, 3:1, 5:1)을 적용한 결과, 대부분의 조합에서 벽면 전력이 자원 할당 비율 변화에 거의 반응하지 않았다.

Table 1. Wall-level power under concurrent execution across resource allocation ratios.

	Workloads	Type	1:1 (W)	2:1 (W)	3:1 (W)	5:1 (W)	Maximum Differences (W)	Maximum Variations
A	ResNet-CPU + GEMM	CPU + GPU	372.4	367.0	365.1	361.8	10.6	±1.4%
B	ResNet-CPU + LLM	CPU + CPU	175.2	173.1	172.0	171.1	4.1	±1.2%
C	GEMM + YOLO	GPU + GPU	370.9	363.4	362.2	362.1	8.8	±1.2%
D	LLM + NodeJS	CPU + CPU	166.6	165.4	164.7	164.9	1.9	±0.6%
E	ResNet-GPU + ResNet-CPU	GPU + CPU	293.5	299.6	298.6	292.5	7.1	±1.2%
F	ResNet-GPU + LLM	GPU + CPU	297.0	300.0	298.2	296.2	3.8	±0.6%
G	ffmpeg + LLM	CPU + CPU	170.0	170.8	170.0	171.1	1.1	±0.3%
H	Training + ffmpeg	GPU + CPU	240.8	293.7	295.3	297.7	56.9	±10.1%
I	LLM-GPU + GEMM	GPU + GPU	385.9	380.6	379.1	380.3	6.8	±0.9%
J	Training + NodeJS	GPU + CPU	295.9	288.2	294.4	294.6	7.6	±1.3%

핵심 발견:

- 10개 조합 중 9개에서 벽면 전력 변동이 **±1.4% 이내**
- 유일한 예외 조합 H(Training + ffmpeg): ±10.1% 변동 → 원인 분석 결과, Training의 데이터 로딩 단계가 CPU 집약적이어서 ffmpeg과 CPU 자원을 경쟁하게 됨 → 3:1 비율에서 ffmpeg에 5코어만 할당되자 CPU 병목으로 Training의 GPU 활용도가 하락
- **결론:** 자원 할당 비율은 시스템 총 전력의 신뢰할 수 있는 대리 지표(proxy)가 되지 못함

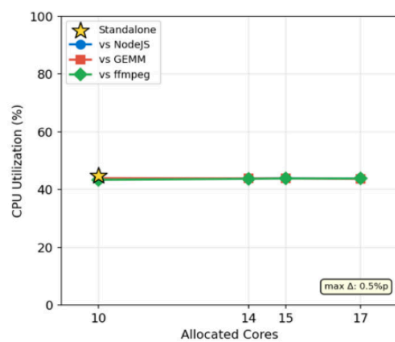
핵심 결과 2: 에너지 프로파일 보존

자원 할당 비율 변화에도 불구하고, 개별 워크로드의 에너지 프로파일(CPU 사용률, GPU 전력 등)이 단독 실행 시와 유사하게 보존되는지 검증하였다.

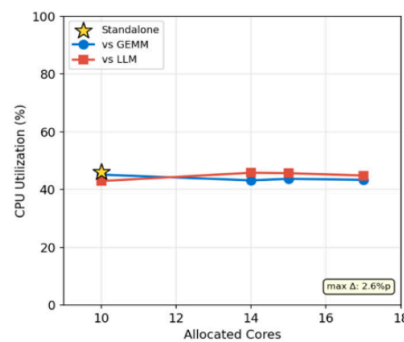
CPU 사용률 보존:

Table 2. Profile preservation under concurrency: standalone vs. concurrent CPU utilization.

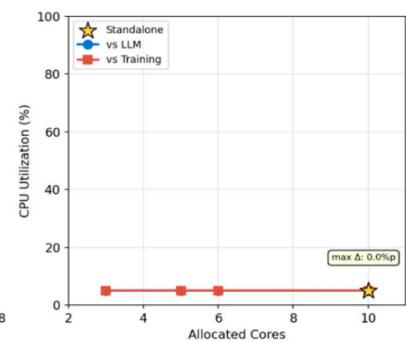
Combination	Workload 1	CPU Utilization			Workload 2	CPU Utilization		
		Standalone	Concurrent	Δ		Standalone	Concurrent	Δ
A	ResNet-CPU	46.0%	45.2%	-0.8	GEMM	5.0%	5.0%	0
B	ResNet-CPU	46.0%	43.0%	-3.0	LLM	44.8%	44.4%	-0.4
C	GEMM	5.0%	5.0%	0	YOLO	5.6%	5.6%	0
D	LLM	44.8%	44.7%	-0.1	NodeJS	5.1%	5.0%	-0.1
E	ResNet-GPU	5.0%	4.8%	-0.2	ResNet-CPU	46.0%	43.6%	-2.4
F	ResNet-GPU	5.0%	5.0%	0	LLM	44.8%	45.2%	0.4
G	ffmpeg	35.3%	36.3%	1	LLM	44.8%	44.3%	-0.5
H	Training	6.5%	3.4%	-3.1	ffmpeg	35.3%	34.7%	-0.6
I	LLM-GPU	4.2%	4.2%	0	GEMM	5.0%	5.0%	0
J	Training	6.5%	6.6%	0.1	NodeJS	5.1%	5.1%	0



(a) LLM



(b) ResNet-CPU



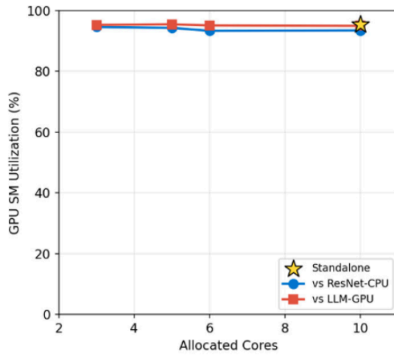
(c) NodeJS

- 20개 워크로드 인스턴스(10 조합 × 2 워크로드) 중 19개가 단독 실행 대비 $\pm 3\%$ p 이내 유지
- 예외: 조합 H의 Training — CPU 병목으로 데이터 로딩 지연 → GPU idle 시간 증가 → SM% 하락

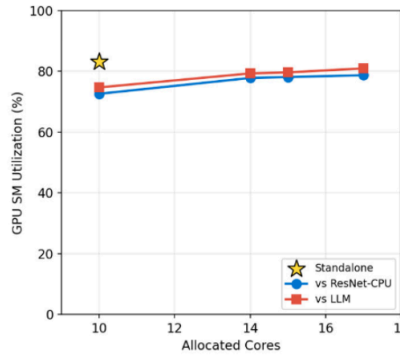
GPU 전력 보존:

Table 3. Profile preservation under concurrency: standalone vs. concurrent GPU power.

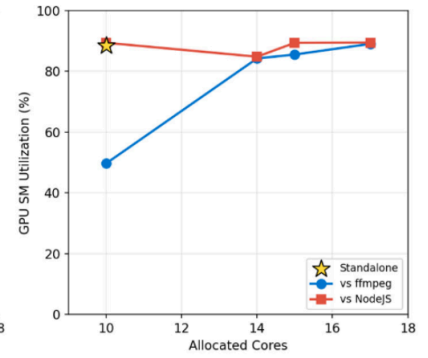
Combination	Workload 1	GPU Power (W)			Workload 2	GPU Power (W)		
		Standalone	Concurrent	Δ		Standalone	Concurrent	Δ
A	ResNet-CPU	N/A	N/A	N/A	GEMM	224.1	223.7	-0.4
B	ResNet-CPU	N/A	N/A	N/A	LLM	N/A	N/A	N/A
C	GEMM	224.1	229.2	5.1	YOLO	43.9	42.2	-1.7
D	LLM	N/A	N/A	N/A	NodeJS	N/A	N/A	N/A
E	ResNet-GPU	162.8	146.6	-16.2	ResNet-CPU	N/A	N/A	N/A
F	ResNet-GPU	162.8	153.2	-9.6	LLM	N/A	N/A	N/A
G	ffmpeg	N/A	N/A	N/A	LLM	N/A	N/A	N/A
H	Training	151.2	99.3	-51.9	ffmpeg	N/A	N/A	N/A
I	LLM-GPU	57.4	56.2	-1.2	GEMM	224.1	227.8	3.7
J	Training	151.2	155.1	3.9	NodeJS	N/A	N/A	N/A



(a) GEMM



(b) ResNet-GPU



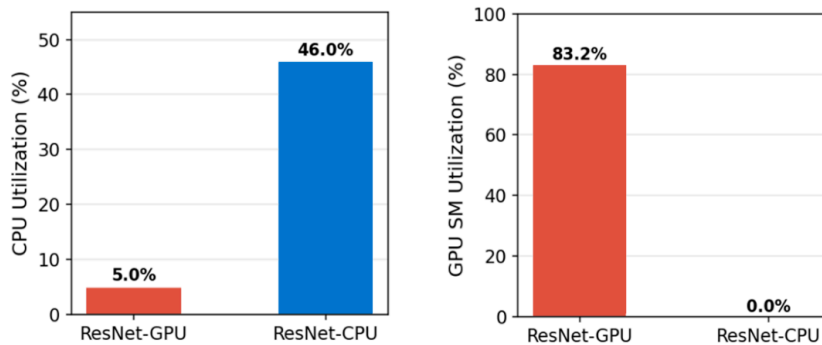
(c) Training

- GEMM의 GPU 전력: 단독 224.1W → 조합 A에서 223.7W (Δ -0.4), 조합 C에서 229.2W (Δ +5.1), 조합 I에서 227.8W (Δ +3.7) → 모두 ± 5.1 W 이내
- 예외: 조합 H의 Training GPU 전력이 단독 151.2W → 99.3W로 하락 (Δ -51.9, CPU 데이터 로딩 병목)

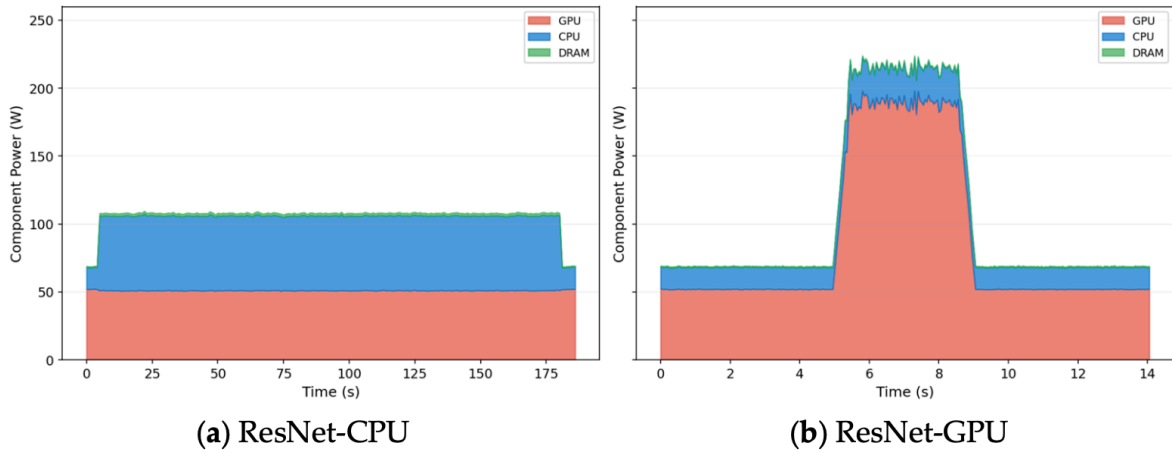
의미: 프로파일 보존은 Track C의 에너지 귀속 모델에서 단독 실행 프로파일을 동시 실행 환경에서의 귀속 기준으로 사용할 수 있는 근거를 제공한다.

핵심 결과 3: 배포 모드에 따른 에너지 전환

동일 모델(ResNet-18)을 CPU와 GPU에서 각각 실행했을 때의 에너지 특성 차이를 분석하였다.



(a) ResNet-GPU vs. ResNet-CPU



핵심 발견:

- GPU 배포 시 순간 전력은 2배이지만, 처리 속도가 56.6배 빨라 **총 에너지는 약 96.5% 감소**
- 에너지 과금 시 순간 전력이 아닌 작업 완료까지의 총 에너지를 기준으로 해야 하는 근거 제공

핵심 결과 4: 에너지 카테고리 분류

논문 Figure 6, 7, 8에 해당하는 에너지 카테고리 분류 결과이다.

Figure 6 — GPU-dominant vs CPU-dominant 분리:

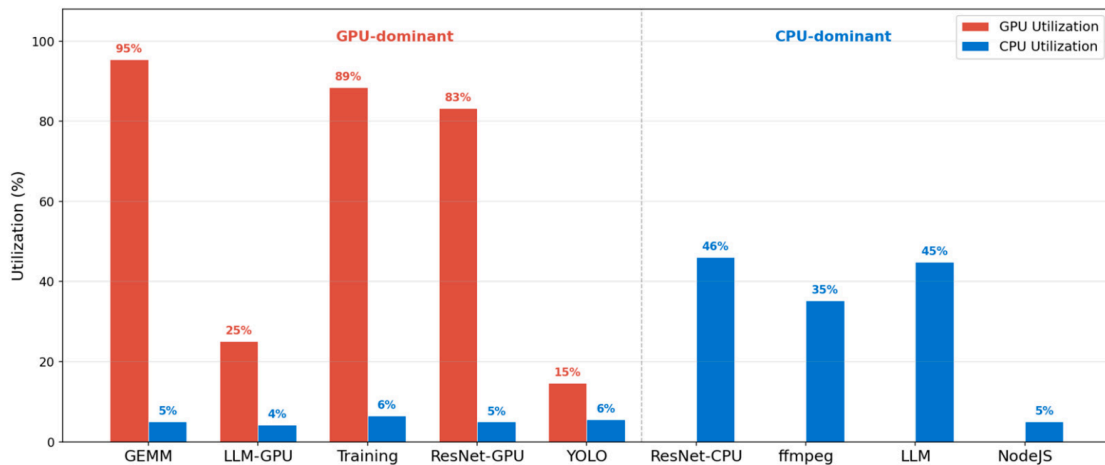


Figure 6. CPU utilization and streaming multiprocessor (SM) utilization across nine workloads.

- GPU SM% > 0인 워크로드: GEMM, ResNet-GPU, Training, LLM-GPU, YOLO → GPU-dominant
- GPU SM% = 0인 워크로드: ResNet-CPU, LLM, NodeJS, ffmpeg → CPU-dominant

- 두 그룹은 에너지 크기와 구성비에서 명확히 구분됨

Figure 7 — GPU-dominant 내부 스펙트럼 (버블 차트):

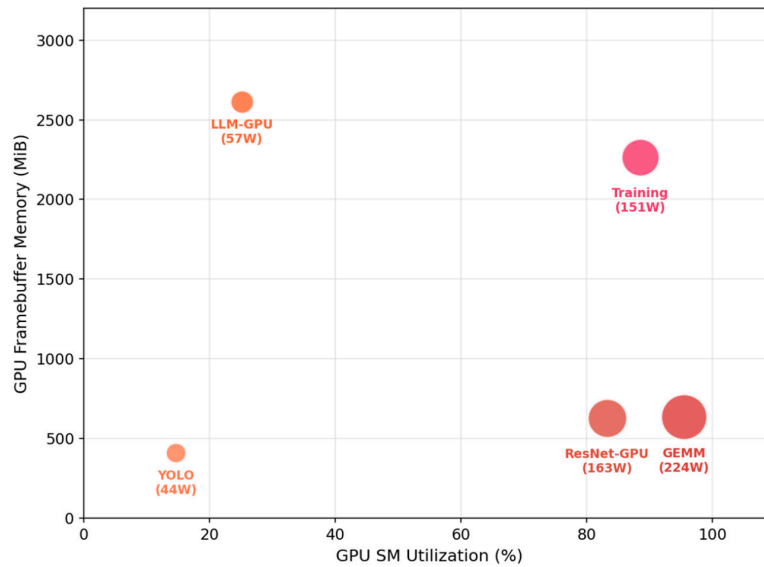


Figure 7. Internal spectrum of GPU-dominant workloads: SM-heavy vs. memory-heavy. Bubble size indicates GPU power consumption.

- X축: SM 사용률(%), Y축: 프레임버퍼 메모리(MiB), 버블 크기: GPU 전력(W)
- SM-heavy 클러스터(우측): GEMM, ResNet-GPU, Training
- Memory-heavy 클러스터(상단): LLM-GPU (낮은 SM, 높은 VRAM)
- Lightweight(좌측 하단): YOLO

Figure 8 — CPU-dominant 내부 스펙트럼 (버블 차트):

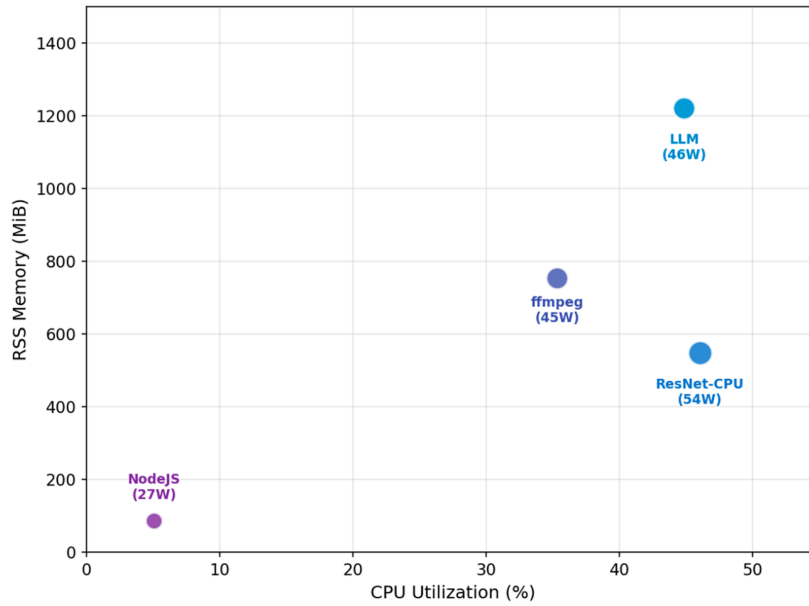


Figure 8. Internal spectrum of CPU-dominant workloads: compute-bound vs. memory bound. Bubble size indicates CPU power consumption.

- X축: CPU 사용률(%), Y축: RSS(MiB), 버블 크기: induced 전력(W)
- Compute-bound: ResNet-CPU (높은 CPU%, 높은 IPC)
- Memory-bound: LLM (높은 CPU%, 낮은 IPC, 높은 RSS)
- Lightweight: NodeJS (낮은 CPU%, 낮은 RSS)

3.5.3.트랙 C: IEEE IoT Journal 논문 실험 결과

Track C의 실험 결과는 IEEE Internet of Things Journal에 제출된 논문의 핵심 발견을 요약한다.

핵심 결과 1: 단독 실행 에너지 프로파일

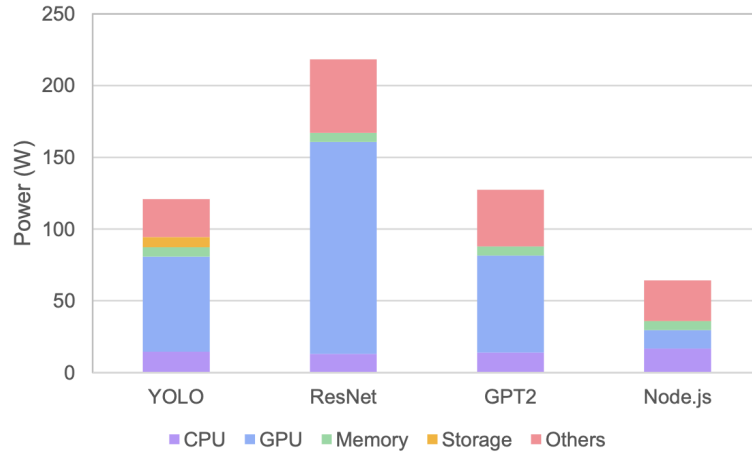


Fig. 4. System-level power breakdown for individual workloads (YOLO, ResNet, GPT2, Node.js). The figure shows total system power and its decomposition into CPU, GPU, memory, storage, and other baseline components when each workload is executed independently.

4개 워크로드를 각각 단독 실행하여 에너지 프로파일 기준선을 확립하였다 (논문 Section IV-B, Figure 4).

워크로드	총 전력 (W)	GPU 비중	CPU 비중	Memory 비중	Others 비중
YOLO	121 W	53%	6%	8%	33%
ResNet-18	218 W	68%	5%	5%	22%
GPT-2	183 W	55%	10%	8%	27%
Node.js	64 W	8%	20%	10%	62%

기준선(idle + Others) 차감 후 워크로드 유발 전력 (논문 Figure 5):

워크로드 유발 전력은 시스템 총 전력에서 idle 기준선과 Others(PSU 손실, 팬 등) 컴포넌트를 제거한 값이다. AI 워크로드의 유발 전력은 65~145W 범위이며, Node.js는 14W이다.

- GPU 전력이 AI 워크로드 유발 전력의 74%(YOLO)~93%(ResNet)를 차지
- Node.js는 유발 전력의 약 94%가 CPU에 귀속
- 메모리 전력은 시스템 수준에서 3~10%이나, 워크로드 수준에서는 약 1%로 감소 (할당 기반 모델 반영)

핵심 발견:

- AI 워크로드의 유발 전력(65~145W)은 Node.js(14W)의 4.8~10.5배 (논문 Section IV-B)
- 동일한 VM 자원(2코어, 4GB)을 할당받더라도 AI 워크로드는 비AI 워크로드 대비 실제 에너지 소비가 크게 다름

핵심 결과 2: 동시 실행 에너지 귀속 정확도

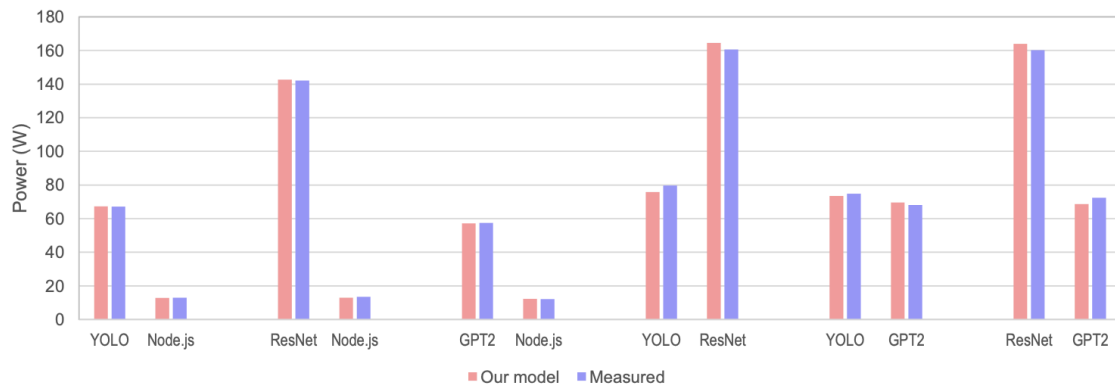


Fig. 8. Comparison between model-attributed workload power under concurrent execution and independently measured workload-induced power from isolated execution.

6개 시나리오에서 에너지 귀속 모델의 정확도를 검증하였다.

시나리오 유형별 귀속 오차 (논문 Section IV-D, Figure 8):

시나리오 유형	조합	AI 워크로드 오차	Non-AI / 상대 워크로드 오차	비고
AI + Non-AI	YOLO/ResNet/GPT2 + Node.js (3개)	0.1%~0.4%	0.4%~4.1%	절대 편차는 유사하나 Node.js는 에너지 작아 상대 오차 큼
AI + AI (비대칭)	YOLO+ResNet, ResNet+GPT2 (2개)	2.4%~5.2% (평균 3.7%)	—	GPU 공유 시 pmon 기반 분리 한계
AI + AI (대칭)	YOLO+GPT2 (1개)	1.8%~2.0%	—	유사 전력 프로파일로 분리 용이

귀속 오차 패턴:

- AI + Non-AI 조합: AI 워크로드는 GPU 에너지가 지배적이고 프로세스별 GPU 측정이 용이하므로 오차가 매우 낮음 (0.1~0.4%)
- AI + Non-AI 조합: Node.js는 절대 에너지가 작아 작은 절대 오차도 상대 비율로는 크게 나타남 (최대 4.1%)
- AI + AI 비대칭 조합: 두 AI 워크로드가 GPU를 공유하므로 pmon 기반 분리의 한계로 오차 증가 (평균 3.7%)
- AI + AI 대칭 조합: 유사한 자원 사용 패턴으로 분리가 비교적 용이 (1.8~2.0%)
- 전체 시나리오에서 최대 오차 약 5% → 에너지 과금 적용에 충분한 정밀도

핵심 결과 3: 과금 불형평성 정량화 (논문 Section IV-D)

모든 동시 실행 실험에서 워크로드들은 동일한 자원(2코어, 4GB 메모리)을 할당받았다. 기존의 할당 기반 과금에서는 동일한 비용이 청구된다. 그러나 실제 에너지 소비는 워크로드 유형에 따라 크게 다르다.

AI + Node.js 조합:

- AI + Node.js 조합에서 에너지 차이는 **4.7~11.0배** (논문 Section IV-D)
- AI 워크로드는 GPU 에너지가 지배적이므로, Node.js 대비 유발 전력이 4.8~10.5배 높은 것이 그대로 반영됨

AI + AI 조합:

조합	에너지 비율	비고
YOLO + ResNet	2.2×	비대칭 — ResNet이 YOLO보다 에너지 소비 큼
ResNet + GPT2	2.4×	비대칭 — ResNet이 GPT2보다 에너지 소비 큼
YOLO + GPT2	≈1× (near-equal)	대칭 — 유사한 에너지 소비

결론:

- 할당 기반 과금(allocation-based pricing): 동일 자원 → 동일 비용 청구
- 에너지 기반 과금(energy-based pricing): AI + Node.js에서 4.7~11.0배, AI + AI에서도 최대 2.4배 차이 반영
- 할당 기반 과금은 실제 에너지 책임을 심각하게 왜곡한다는 것이 본 연구의 핵심 주장

3.6 논문 게재 현황

본 프로젝트는 연구 트랙으로서, 수행 결과를 국제 학술지 논문으로 게재하는 것을 목표로 한다. 현재까지의 논문 게재 현황은 다음과 같다.

3.6.1.트랙 B: MDPI Applied Sciences (게재 완료)

항목	내용
논문 제목	Energy Behavior of AI Workloads Under Resource Partitioning in Multi-Tenant Systems
저자	Jiyeon Kim, Siyeon Kang, Woorim Shin, Kyungwoon Cho, and Hyokyung Bahn
학술지	MDPI Applied Sciences (Appl. Sci. 2026, 16, 3129)
DOI	https://doi.org/10.3390/app16073129
접수일	2026년 3월 3일

승인일	2026년 3월 22일
게재일	2026년 3월 24일
상태	게재 완료 (Published)
GitHub	https://github.com/Team37-OptimusPrime/energy-partitioning-paper

논문 주요 기여:

1. 자원 할당 비율 변화가 시스템 총 전력에 미치는 영향이 미미함을 실증 ($\pm 1.4\%$ 이내)
2. 개별 워크로드의 에너지 프로파일이 동시 실행 환경에서도 보존됨을 확인
3. AI 워크로드의 에너지 카테고리 분류 프레임워크 제안 (GPU-dominant/CPU-dominant)

3.6.2. 트랙 C: IEEE Internet of Things Journal (심사 중)

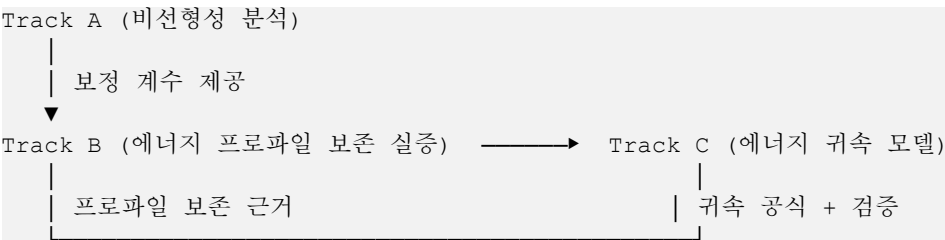
항목	내용
논문 제목	An Energy Cost Model for AI Workloads under Shared Resource Environments
저자	Woorim Shin, Jiyeon Kim, Siyeon Kang, Kyungwoon Cho, and Hyokyung Bahn
학술지	IEEE Internet of Things Journal
상태	투고 완료 (Under Review)
GitHub	https://github.com/Team37-OptimusPrime/vm-power-attribution

논문 주요 기여:

1. CPU/GPU/메모리/스토리지 4개 컴포넌트별 에너지 분해 모델 설계
2. 이중 구조(할당 기반 + 사용량 기반) GPU 에너지 귀속 방법 제안
3. 6개 시나리오에서 최대 약 5% 이내의 귀속 오차 달성
4. 할당 기반 과금의 불형평성을 정량적으로 입증 (AI vs Non-AI: 4.7~11.0× 차이)
5. 다른 서버 환경(Alienware Aurora R12)에서 모델의 일반화 가능성 검증

3.6.4 트랙 간 연계 및 프로젝트 전체 기여

세 트랙의 연구는 다음과 같이 유기적으로 연결된다:



↓
VM별 에너지 과금 모델 완성

- **Track A → B/C:** 자원 사용률과 전력의 비선형 관계를 정밀하게 모델링하여, Track B/C의 선형 비례 귀속 공식의 오차를 보정
- **Track B → C:** 다중 워크로드 환경에서 개별 워크로드의 에너지 프로파일이 보존된다는 실증 결과가 Track C의 귀속 모델에서 단독 실행 프로파일을 기준선으로 사용하는 이론적 근거 제공
- **Track C → 프로젝트 목표:** 최종적으로 VM별 에너지 사용량을 5% 이내 정밀도로 귀속하는 모델을 완성하여, 에너지 기반 클라우드 과금 체계의 기술적 기반 마련